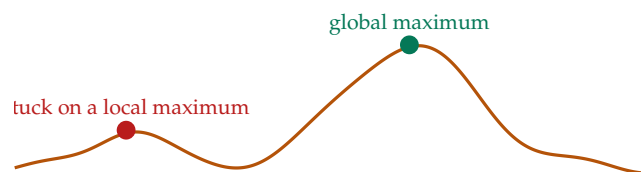

Artificial Intelligence: Search Methods for Problem Solving

Week 3 — Heuristic Search and Escaping Local Optima

Detailed study notes built on the lectures of Prof. Deepak Khemani (IIT Madras)

IITM BS in Programming & Data Science



Part I follows the *Heuristic Search* deck; Part II follows the *Escaping Local Optima (Deterministic Local Search)* deck. Yellow boxes give intuition, blue boxes are definitions, purple boxes are formulas and derivations, black boxes are algorithms (as on the slides), green boxes are side notes, red boxes flag exam traps and slide errata. Every number in the worked examples was checked by running code.

Contents

Part I — Heuristic Search	3
1 From Blind Search to a Sense of Direction	3
1.1 Recap: DFS and BFS are blind	3
1.2 The pull of gravity	3
1.3 A rolling stone	3
1.4 The idea of a heuristic function	4
2 Best First Search	5
2.1 OPEN becomes a priority queue	5
2.2 The algorithm	5
2.3 A worked trace	6
2.4 Heuristics for the 8-puzzle	7
2.5 Heuristics for geographical route finding	8
2.6 Heuristics can mislead: a real map	9
2.7 Search frontiers and completeness	9
2.8 Quality of solution	10
3 Hill Climbing and the Terrain Defined by Heuristics	11
3.1 Hill Climbing: a local search algorithm	11
3.2 Hill Climbing is a constant-space algorithm	11
3.3 Steepest gradient ascent	12
3.4 8-puzzle: heuristic values along a real solution	12
3.5 The Rubik's cube: fluctuating heuristic values	13
3.6 A blocks world problem	13
3.7 Heuristic functions <i>define</i> the terrain	15
3.8 Escaping local optima	16
Part II — Escaping Local Optima	16
4 Solution Space Search: SAT and TSP	16
4.1 Recap and plan	16
4.2 Solution space search	16
4.3 Synthesis vs perturbation	17
4.4 The SAT problem	17
4.5 Solution space search and perturbative methods	18

4.6	The Travelling Salesperson Problem	18
4.7	Computational complexity and exploding search spaces	20
5	Deterministic Methods for Escaping Local Optima	22
5.1	Exploitation vs exploration	22
5.2	Beam Search	22
5.3	Different neighbourhood functions	23
5.4	Variable Neighbourhood Descent (VND)	24
5.5	Best Neighbour: move anyway	25
5.6	Tabu Search: not allowed back immediately	25
5.7	A 4-variable SAT landscape	26
5.8	Iterated Hill Climbing	29
	Practice: Test Yourself	31
	Part A — Numerical problems and traces	31
	Part B — Subjective questions	34
	Part C — MCQs	37

Part I — Heuristic Search

Module 1 From Blind Search to a Sense of Direction

1.1 Recap: DFS and BFS are blind



Slide erratum: The BFS recap slide

As in Week 2, the BFS recap slide says BFS “adds new candidates at the *head* of OPEN”. BFS appends them at the **tail** (queue); only DFS adds at the head (stack). Both pick from the head.

Definition: Blind / uninformed search

Both DFS and BFS are **oblivious of the goal**. Irrespective of where the goal is in the state space, they set out on the same predetermined trajectories every time. What is needed is a search algorithm with a **sense of direction**.

1.2 The pull of gravity

The gradient on a mountainside gives a sense of direction to water flowing down a stream: water always takes the **steepest gradient descent** path. A physical agent wanting to reach the bottom of a valley can do the same:

- test the landscape by “taking a step” in each direction;
- choose the direction with the steepest gradient.

The two questions this raises for search are: **what defines the landscape** we are traversing, and **how is the gradient defined**? The answer is a heuristic function.

1.3 A rolling stone

Picture a stone rolling down a two-dimensional landscape. It has mass (so it feels gravity) but **no momentum**. Where will it finally rest?

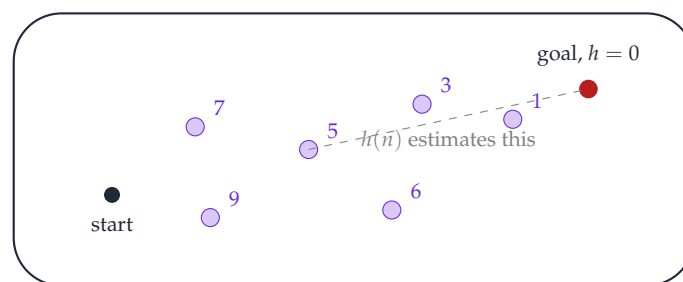


Intuition: Local search

The stone tests only its *immediate* surroundings and follows the steepest gradient. Without momentum it stops in the first dip it meets, even if a much deeper valley lies just over the next rise. This is **local search**: it looks only at its immediate neighbourhood. Everything good and bad about hill climbing (Module 3) is already in this picture.

1.4 The idea of a heuristic function**Definition: Heuristic function $h(N)$**

A function that takes a state (node) N and returns a number that is an **estimate of the distance of N from the goal**. It is typically **user defined**, in addition to MoveGen and GoalTest. Since $h(\text{goal}) = 0$, search effectively seeks to **minimise** $h(N)$.

**Formula / Derivation: What a heuristic changes in the search algorithm**

The domain description grows from two functions to three:

$$\langle s_0, \text{MOVEGEN} : S \rightarrow 2^S, \text{GOALTEST} : S \rightarrow \{T, F\}, h : S \rightarrow \mathbb{R}_{\geq 0} \rangle, \\ h(g) = 0 \text{ for every goal } g.$$

The algorithm now answers the Week 2 key question — *which node to pick from OPEN?* — with: **the node with the lowest h** . Informally, h defines the *landscape*; comparing h of neighbours defines the *gradient*.

Note: Heuristic = estimate, not truth

If h were exactly the true distance to the goal, search would walk straight to it. Real heuristics are cheap approximations computed from the state alone (count misplaced tiles, straight-line distance, ...). How good the approximation is determines how well the search is guided. Minimising an estimate of distance and maximising an estimate of “goodness” are the same idea with the sign flipped; the slides use both (8-puzzle: minimise; blocks world and SAT: maximise).

Module recap

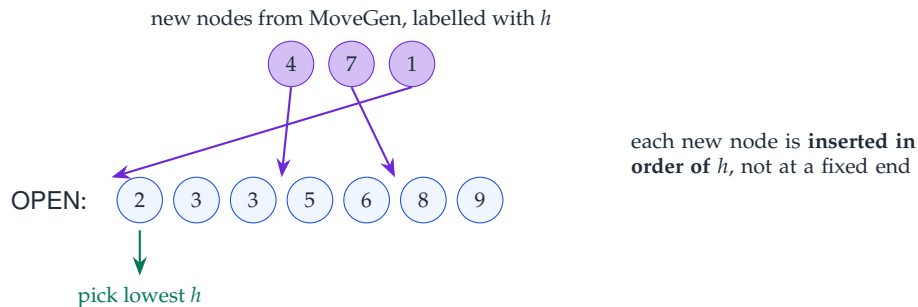
- DFS and BFS are blind: same trajectory wherever the goal is.
- Water and rolling stones follow the steepest gradient; a search can do the same if we define a landscape.
- The heuristic $h(N)$ estimates the distance from N to the goal ($h(\text{goal}) = 0$); it is supplied by the user alongside MoveGen and GoalTest.
- Local search looks only at the immediate neighbourhood — and can stop in a local dip.

Module 2 Best First Search

2.1 OPEN becomes a priority queue

Definition: Best First Search

Best First Search inserts new candidates into OPEN **sorted on $h(n)$** , so OPEN is a **priority queue**. It always picks the candidate at the head of OPEN — the node with the **lowest** heuristic value, i.e. the one that *looks* closest to the goal.



2.2 The algorithm

The slide takes the Week 2 DFS pseudocode, strikes out “Depth” and changes one line:

DFS	$\text{OPEN} \leftarrow \text{append}(\text{NEW}, \text{tail OPEN})$
Best First	$\text{OPEN} \leftarrow \text{sort}_h(\text{append}(\text{NEW}, \text{tail OPEN}))$
Best First, better	$\text{OPEN} \leftarrow \text{merge}(\text{sort}_h(\text{NEW}), \text{tail OPEN})$

In practice OPEN is maintained as a **priority queue**. The nodePair becomes a **triple** that includes $h(n)$.

Algorithm: Best-First-Search(S) — from the slide

```

1: OPEN  $\leftarrow (S, \text{null}, h(S)) : []$ 
2: CLOSED  $\leftarrow$  empty list
3: while OPEN is not empty do
4:   nodePair  $\leftarrow$  head OPEN
5:    $(N, \_, \_)$   $\leftarrow$  nodePair ▷ a triple, even if we still call it a nodePair
6:   if GOALTEST( $N$ ) = TRUE then
7:     return RECONSTRUCTPATH(nodePair, CLOSED)
8:   else CLOSED  $\leftarrow$  nodePair : CLOSED
9:   children  $\leftarrow$  MOVEGEN( $N$ )
10:  newNodes  $\leftarrow$  REMOVESEEN(children, OPEN, CLOSED)
11:  newPairs  $\leftarrow$  MAKEPAIRS(newNodes,  $N$ ) ▷ each with its  $h$  value
12:  OPEN  $\leftarrow$   $\text{sort}_h(\text{newPairs} ++ \text{tail OPEN})$ 
13:  end if
14: end while
15: return empty list

```

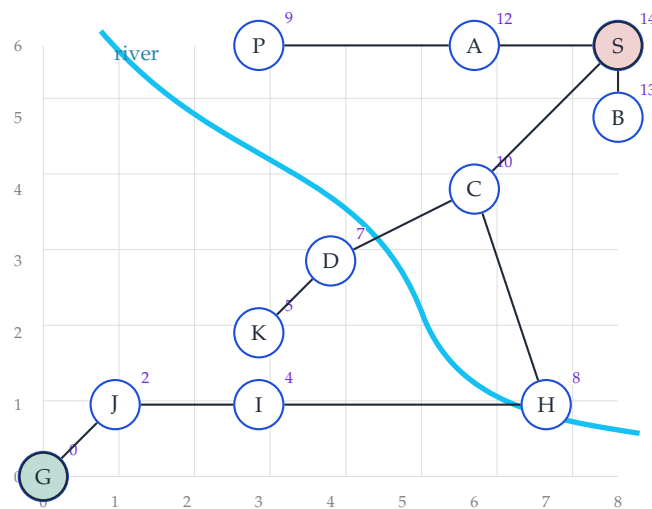
Formula / Derivation: Cost of maintaining OPEN: sort vs merge vs heap

Let $|\text{OPEN}| = m$ and b new children.

- Re-sorting the whole list each time: $O((m + b) \log(m + b))$ per expansion.
- Sorting only the b children and *merging* with the already-sorted tail: $O(b \log b + m)$.
- A binary-heap priority queue: $O(\log m)$ per insertion and per removal of the minimum, so $O(b \log m)$ per expansion. This is what real implementations use (e.g. Python's `heapq`).

2.3 A worked trace

A small road map with the goal at the origin. Each node is labelled with its coordinates, and h is the Manhattan distance to G, $h(x, y) = |x| + |y|$.



Step	Pick	OPEN after the step (node:h), sorted	CLOSED
0		[S:14]	[]
1	S	[C:10, A:12, B:13]	[S]
2	C	[D:7, H:8, A:12, B:13]	[C, S]
3	D	[K:5, H:8, A:12, B:13]	[D, C, S]
4	K	[H:8, A:12, B:13] dead end	[K, D, C, S]
5	H	[I:4, A:12, B:13]	[H, K, D, C, S]
6	I	[J:2, A:12, B:13]	[I, H, ...]
7	J	[G:0, A:12, B:13]	[J, I, ...]
8	G	goal ; path S-C-H-I-J-G	

Intuition: What the trace shows

Best First heads straight towards the goal (C, then D), but the region around D is a **dead end** on this side of the river. Because OPEN still holds H, the search simply *resumes* from the next best candidate — it does not have to backtrack step by step as DFS would. It never expands A, B or P, which point away from the goal. The slide's own example (next page) shows exactly this pattern: it first hits a dead end (steps 2–4), then recovers.

2.4 Heuristics for the 8-puzzle

Definition: Two heuristics for sliding-tile puzzles

- $h_1(n)$ = number of tiles out of place — the **Hamming distance** (the blank is not counted).
- $h_2(n) = \sum_{\text{tiles}}$ Manhattan distance of each tile to its destination — the **Manhattan distance**.

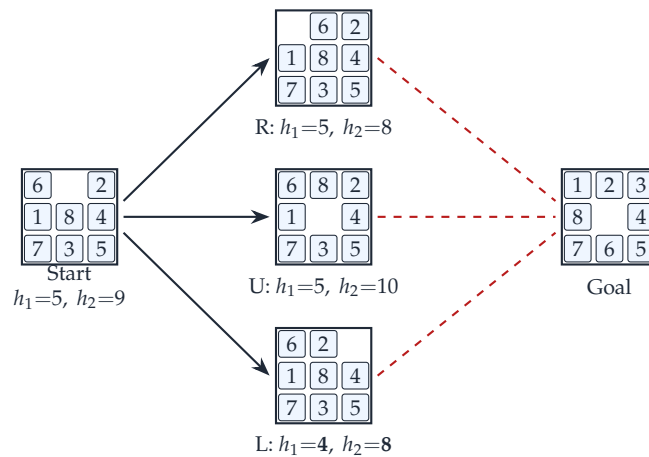
Formula / Derivation: Manhattan distance of one tile

If tile t is at row r , column c and its goal position is (r^*, c^*) , it needs at least

$$d_t = |r - r^*| + |c - c^*|$$

moves (each move shifts it by one cell horizontally or vertically). Summing over tiles gives $h_2 = \sum_t d_t$. Since every tile out of place has $d_t \geq 1$, we always have $h_2 \geq h_1$: Manhattan is the more *informed* of the two.

2.4.1 Which state is closest to the goal?



The slide leaves the question open; the values above were computed for these notes. **L** is best by both heuristics (Hamming 4; Manhattan 8, tied with R). **U** is worst by Manhattan: it moves the 8 from its home row.

Formula / Derivation: Computing h_2 for the start state, tile by tile

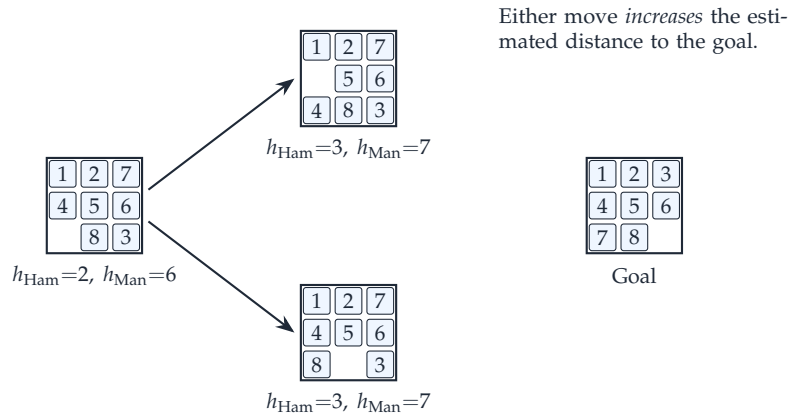
Goal positions (row, column): 1:(1,1) 2:(1,2) 3:(1,3) 8:(2,1) 4:(2,3) 7:(3,1) 6:(3,2) 5:(3,3).

Tile	now at	goal	d_t
6	(1,1)	(3,2)	$2 + 1 = 3$
2	(1,3)	(1,2)	$0 + 1 = 1$
1	(2,1)	(1,1)	$1 + 0 = 1$
8	(2,2)	(2,1)	$0 + 1 = 1$
4	(2,3)	(2,3)	0
7	(3,1)	(3,1)	0
3	(3,2)	(1,3)	$2 + 1 = 3$
5	(3,3)	(3,3)	0
$h_2 =$			9

Tiles out of place: 6, 2, 1, 8, 3, so $h_1 = 5$.

2.4.2 A local minimum

With goal 1 2 3 / 4 5 6 / 7 8 _:



Definition: Local minimum (of h)

A state all of whose neighbours have a *larger* heuristic value. A search that only ever moves to better neighbours stops here, even though the goal is still far away (tiles 3 and 7 must swap places, which takes many moves).

Intuition: Similarity is the inverse of distance

The slide redraws this with a photograph of a puppy cut into tiles. Either move makes the picture look *less* like the goal picture, yet you must make one of them to solve the puzzle. A human solving the Rubik's cube faces the same thing: to finish, you must temporarily scramble parts you have already solved. Measuring "how similar is this to the goal" is a heuristic, and heuristics can be misleading.

2.5 Heuristics for geographical route finding

Formula / Derivation: Euclidean and Manhattan distance

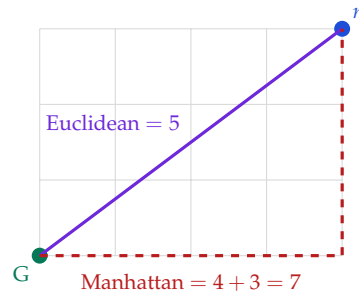
For a node at (x_n, y_n) and the goal at (x_G, y_G) :

$$h_{\text{Euclid}}(n) = \sqrt{(x_G - x_n)^2 + (y_G - y_n)^2}, \quad h_{\text{Manhattan}}(n) = |x_G - x_n| + |y_G - y_n|.$$

With the goal at the origin, the slide's three neighbours of S are at $(6,6)$, $(8,5)$, $(6,4)$:

Node	Euclidean	Manhattan (city block)
$(6,6)$	$\sqrt{72} = 8.49$ (slide: 8.42)	12
$(8,5)$	$\sqrt{89} = 9.43$	13
$(6,4)$	$\sqrt{52} = 7.21$	10

Both heuristics rank the three the same way: $(6,4)$ first. (The slide's 8.42 is a small rounding slip; $\sqrt{72} = 8.485$.) Always $h_{\text{Euclid}} \leq h_{\text{Manhattan}} \leq \sqrt{2} h_{\text{Euclid}}$.



2.5.1 Best First Search has a sense of direction

The slide runs Best First Search on a road map with a river. The order in which nodes are picked (with their h values) is:

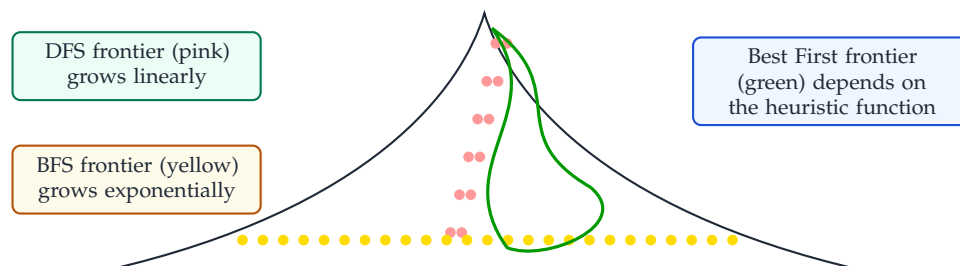
$$S \rightarrow 10 \rightarrow 7 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 6 \rightarrow 4 \rightarrow 2 \rightarrow G.$$

It first hits a **dead end** (steps 2–4: the nodes with $h = 7, 7, 8$ sit in a pocket on the start's side of the river with no crossing), then climbs back through the node with $h = 9$ to cross the river and reach the goal. Nodes on OPEN that were never picked ($h = 12, 13, 8, 10, 4$) show where the search chose *not* to go.

2.6 Heuristics can mislead: a real map

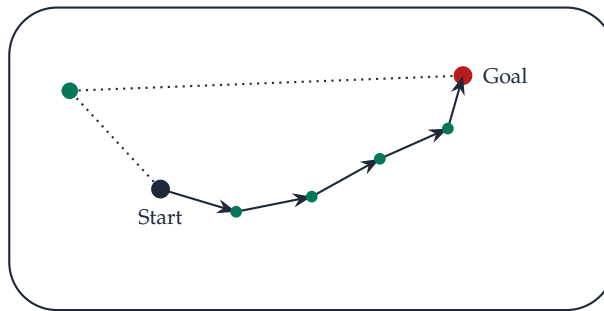
The slides show Google Maps directions from IIT Madras to the Anna Alumni Club, and the underlying road graph. The straight-line heuristic pulls the search towards the nodes nearest the goal as the crow flies — but those roads lead to the Adyar river with no bridge. The search is driven towards these nodes and then **has to move away** to find the real route, which goes around.

2.7 Search frontiers and completeness



- **Space.** The frontier is an indication of space requirement. For Best First it depends on how good h is: with a perfect heuristic it is as small as DFS's; with a useless one it can be as bad as BFS's.
- **Completeness?** On a *finite* graph, Best First Search with CLOSED is complete: it never discards a candidate, so if the heuristic leads it astray it eventually comes back to the nodes still on OPEN. On an infinite graph a misleading heuristic can lure it down an infinite path, so it is not complete in general.

2.8 Quality of solution



Best First may choose the 5-move path because every node on it *looks* closer to the goal, while a 2-move path via a node that looks far away exists.

Exam trap: Best First is not guaranteed to find the shortest path

Best First Search chooses nodes by h alone — how close they *seem* to the goal — and ignores how far they are from the start. It can therefore return a longer path than necessary. Fixing this, by also counting the cost already incurred, is the idea behind the A* algorithm later in the course.

Module recap

- Best First Search: OPEN sorted on h (priority queue), pick the lowest h . One-line change from DFS: $\text{OPEN} \leftarrow \text{sort}_h(\text{newPairs} ++ \text{tail OPEN})$.
- 8-puzzle heuristics: Hamming h_1 (tiles out of place) and Manhattan h_2 (sum of tile distances); $h_2 \geq h_1$. Both have local minima.
- Route finding: Euclidean $\sqrt{\Delta x^2 + \Delta y^2}$ and Manhattan $|\Delta x| + |\Delta y|$.
- Best First has a sense of direction, can recover from dead ends because OPEN is kept, is complete on finite graphs, but does not guarantee the shortest path; its space depends on h .

Module 3 Hill Climbing and the Terrain Defined by Heuristics

3.1 Hill Climbing: a local search algorithm

Definition: Hill Climbing

Move to the **best neighbour** if it is **better** than the current node; otherwise terminate.

Algorithm: Hill Climbing — from the slide (minimising h)

```

1:  $node \leftarrow Start$ 
2:  $newNode \leftarrow \text{head}(\text{sort}_h(\text{MOVEGEN}(node)))$ 
3: while  $h(newNode) < h(node)$  do
4:    $node \leftarrow newNode$ 
5:    $newNode \leftarrow \text{head}(\text{sort}_h(\text{MOVEGEN}(node)))$ 
6: end while
7: return  $node$ 

```

Note: Three things to notice on the slide

1. **In practice sorting is not needed**, only the best node: a single pass computing the minimum costs $O(b)$ instead of $O(b \log b)$.
2. **Change of termination criterion.** Best First stops when GoalTest succeeds. Hill Climbing stops when *no neighbour is better*. It treats the problem as an **optimisation** problem: find a node where h cannot be improved locally.
3. **Hill Climbing has burnt its bridges by not storing OPEN.** There is no list of alternatives to fall back on: once it moves, the other children are forgotten.

Best First Search

keeps OPEN (all candidates)
picks the globally best candidate on OPEN
can jump back to an old candidate
stops when the goal is found

Hill Climbing

keeps only the current node
picks the best *neighbour*
never goes back
stops when no neighbour is better

3.2 Hill Climbing is a constant-space algorithm

Formula / Derivation: Space and time

Space. HC stores the current node and examines its b neighbours: $O(b)$, independent of depth — **constant** in the size of the search. A vast improvement on the exponential space of BFS (and on the CLOSED list that DFS keeps).

Time. Each step strictly improves h . If h takes integer values from $h(s_0)$ down to at least 0, there can be at most $h(s_0)$ steps. More generally the path never revisits a node, so the number of steps is at most the length of the path it walks: **linear**.

The price. It is **not complete**, and it may stop at a local optimum that is not the global optimum (which corresponds to the solution).

3.2.1 Hill Climbing on the Module 2 map

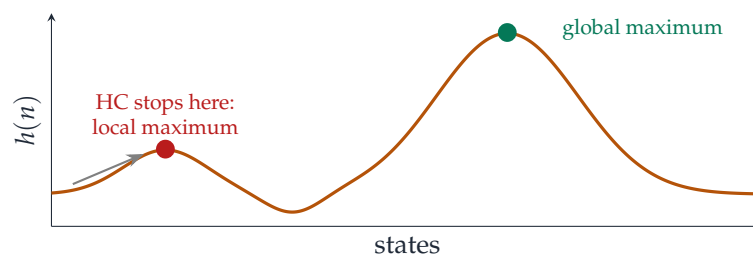
Same graph and Manhattan heuristic as the Best First trace:

$$S(14) \rightarrow C(10) \rightarrow D(7) \rightarrow K(5) \rightarrow \text{stop.}$$

K's only neighbour is D with $h = 7 > 5$, so K is a **local minimum**. Best First recovered from this dead end because H was still on OPEN; Hill Climbing has thrown H away and **fails**.

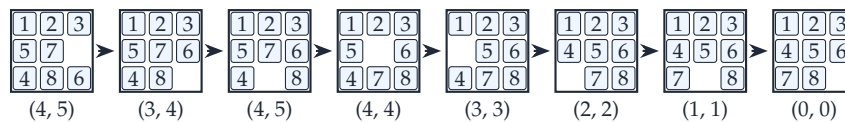
3.3 Steepest gradient ascent

For a **maximisation** problem (the slides' blocks world and SAT), the same algorithm is *steepest gradient ascent*: climb in the direction that increases h most; stop at the top. The cartoon shows a blindfolded climber who can only feel the slope under his feet: on the left he steps uphill, on the right he is on a summit — every direction goes down — so he stops, satisfied.

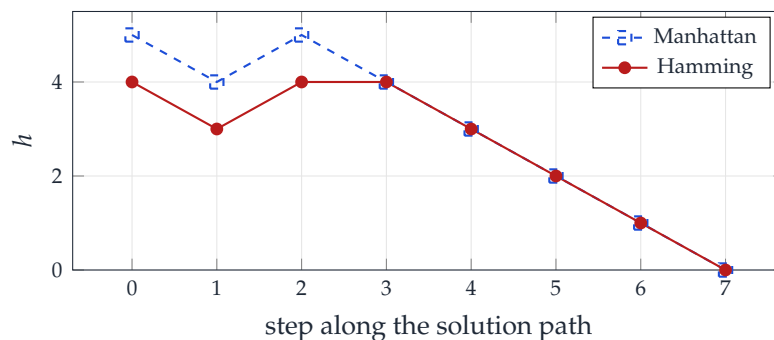


3.4 8-puzzle: heuristic values along a real solution

Goal 1 2 3 / 4 5 6 / 7 8 _ . A 7-move solution and its heuristic values (checked by computer):



Under each board: (Hamming, Manhattan).

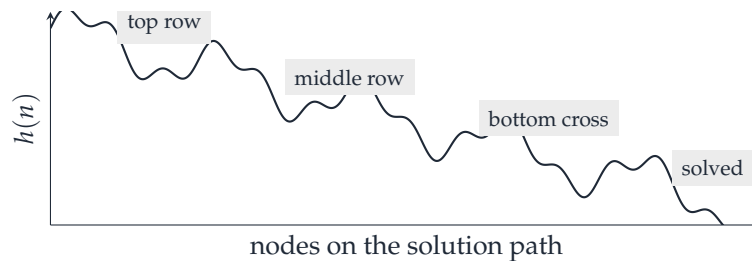


Intuition: What the plot says

Along a *correct* solution, the heuristic goes **up** at step 2 (both heuristics) and stays flat at step 3 (Hamming). A hill climber at step 1 would refuse to make move 2, because it looks worse. So even when a solution is only 7 moves away, strict Hill Climbing can fail. The Manhattan values are never below the Hamming values, as the formula $h_2 \geq h_1$ predicts.

3.5 The Rubik's cube: fluctuating heuristic values

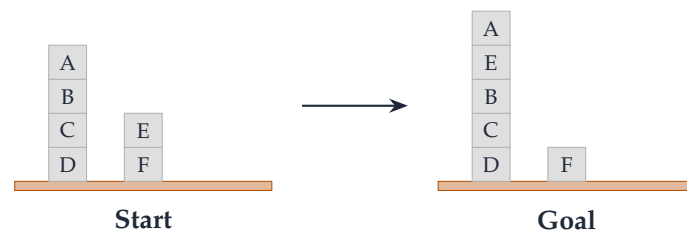
For human-generated solutions of the Rubik's cube (solve the top row, then the middle row, then the bottom cross, and so on), a simple heuristic such as “number of stickers in the right place” passes through **many local minima**. Each stage of the human method temporarily breaks pieces that were already right, in order to place new ones.



3.6 A blocks world problem

Definition: Blocks world

Blocks sit on a table or on each other. A move takes a *clear* block X (nothing on top) and puts it at location Y (the table or on top of another clear block). **Note: the slides treat this as a maximisation problem** — a higher h is better.



3.6.1 Heuristic h_1 : is each block on the right thing?

Formula / Derivation: h_1

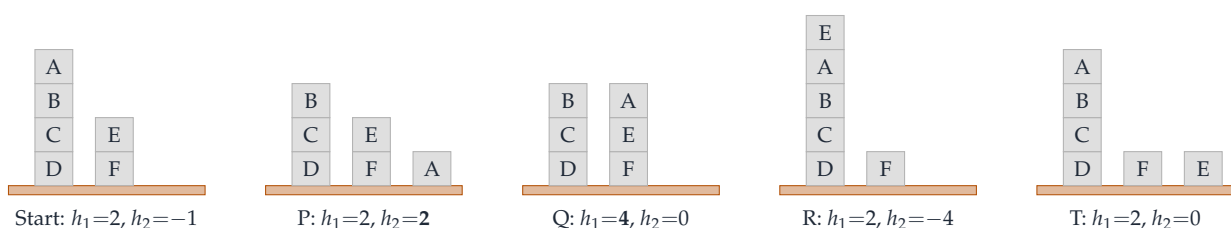
For every block: +1 if it is on the correct block/table (as in the goal), -1 if it is on a wrong block/table.

$$h_1(n) = val_A + val_B + val_C + val_D + val_E + val_F, \quad h_1 \in [-6, 6], \quad h_1(\text{goal}) = 6.$$

Start: A on B (goal: on E) -1; B on C +1; C on D +1; D on table +1; E on F (goal: on B) -1; F on table +1:

$$h_1(S) = (-1) + 1 + 1 + 1 + (-1) + 1 = 2.$$

The start has four moves (A to the table, A onto E, E to the table, E onto A):



State	val_A	val_B	val_C	val_D	val_E	val_F	h_1
S (start)	-1	1	1	1	-1	1	2
P (A to table)	-1	1	1	1	-1	1	2
Q (A onto E)	1	1	1	1	-1	1	4
R (E onto A)	-1	1	1	1	-1	1	2
T (E to table)	-1	1	1	1	-1	1	2

h_1 thinks moving to **Q** is the best idea, because in **Q** block A is on block E (as in the goal). But from **Q** every move is worse: B to table (**U**), B onto A (**V**), A to table (**P**), A back onto B (**Start**) all have $h_1 = 2$. **From h_1 's perspective Q is a maximum** — a local one, since the goal has $h_1 = 6$. Hill Climbing with h_1 gets stuck at **Q**.

Intuition: Why h_1 is fooled

h_1 judges each block only by what it rests on. In **Q**, A is on E “correctly”, but E itself sits on F, the wrong block — and to fix E you must first move A off it again. h_1 rewards a locally correct placement on top of a globally wrong structure.

3.6.2 Heuristic h_2 : is each block on the right structure?

Formula / Derivation: h_2

For every block X, let n be the number of blocks in the structure below X (the tower from X down to the table).

$$val_X = \begin{cases} +n & \text{if X and every block below it are exactly as in the goal,} \\ -n & \text{otherwise.} \end{cases} \quad h_2(n) = \sum_X val_X.$$

Start: A has 3 blocks below it (B, C, D) and is misplaced: -3. B has 2 below, correct: +2. C: +1. D on the table: 0. E has 1 below (F), wrong: -1. F: 0.

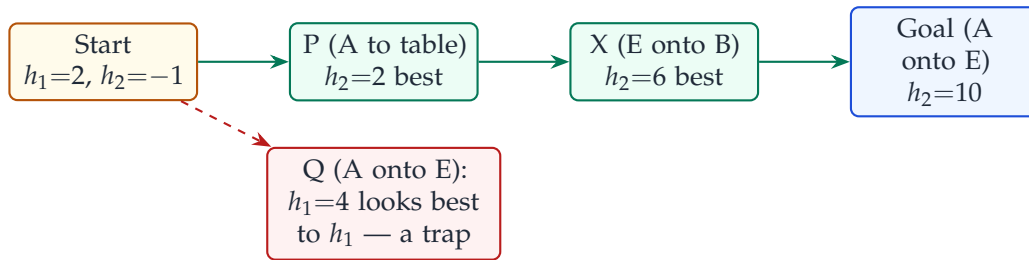
$$h_2(S) = -3 + 2 + 1 + 0 - 1 + 0 = -1.$$

State	A	B	C	D	E	F	h_2 (this definition)	slide
S	-3	2	1	0	-1	0	-1	-1
P (A to table)	0	2	1	0	-1	0	2	6
Q (A onto E)	-2	2	1	0	-1	0	0	2
R (E onto A)	-3	2	1	0	-4	0	-4	-4
T (E to table)	-3	2	1	0	0	0	0	0
X (from P: E onto B)	0	2	1	0	+3	0	6	10
W (from P: E onto A)	0	2	1	0	-1	0	2	2
Goal	+4	2	1	0	+3	0	10	16

Slide erratum: The slides' h_2 values are internally inconsistent

The slides give $h_2(\text{goal}) = 16$, $h_2(S) = -1$, $h_2(P) = 6$, $h_2(Q) = 2$, $h_2(R) = -4$, $h_2(T) = 0$, $h_2(X) = 10$, $h_2(W) = 2$. No single reading of the definition reproduces all eight (checked by trying every natural variant by computer). Counting only the blocks below (as above) matches S, R, T, W exactly. Counting the table as one more element of every structure matches Q, X and the goal (it gives $S = 1$, $P = 4$, $Q = 2$, $R = -2$, $T = 2$, $X = 10$, $W = 4$, $\text{goal} = 16$). **The conclusions are identical under both:** from the start, P is the unique best move; from P, X is

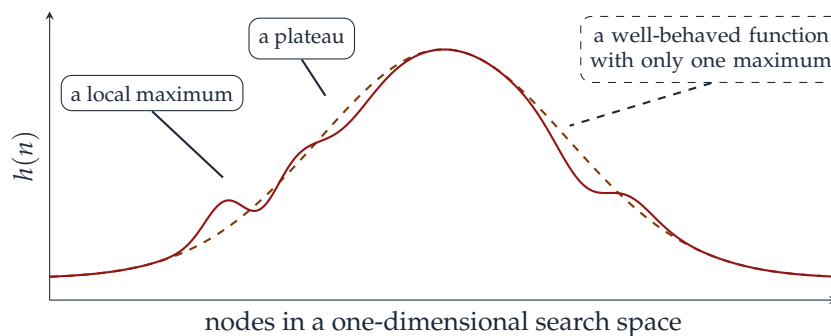
the unique best move. For an exam, reproduce the slide's reasoning and ranking; if asked to compute, state which convention you use.



Intuition: Why h_2 is more discriminative

h_2 rewards a block only if the *whole tower beneath it* is right, and it weights deep correct structure more heavily. Moving A to the table (P) takes A off a wrong three-block structure, which h_2 values (from -3 to 0), while h_1 sees no difference. From P, putting E on B builds a correct four-block tower (worth $+3$ for E). Then A onto E completes the goal. **A more informed heuristic makes Hill Climbing succeed where a naive one fails.** The slide notes that three other moves from P (moving B) are bad and not drawn.

3.7 Heuristic functions *define* the terrain



Definition: Features of a heuristic landscape

- **Global maximum (optimum):** the best value anywhere; for a well-designed heuristic it is the goal.
- **Local maximum:** better than all its neighbours, but not globally best. Hill Climbing stops there.
- **Plateau:** a flat region where neighbours have the *same* value. There is no gradient to follow; strict Hill Climbing stops.
- **Ridge:** a sequence of equal-valued nodes, each of which has neighbours only equal or worse, that nevertheless leads up to a better region (Part II shows one in a SAT landscape).

Intuition: The same problem, different terrains

The state space is fixed by the problem; the *terrain* is created by the heuristic you choose. h_1 and h_2 on the blocks world are the same states on two different landscapes: h_1 's has a local maximum at Q, h_2 's does not (on this path). A *monotonic, well-behaved* heuristic — one maximum, no plateaus — would make Hill Climbing perfect. Such heuristics are hard to find, which is why Part II looks for algorithms that do better than Hill Climbing.

3.8 Escaping local optima

Given that it is difficult to define heuristic functions that are monotonic and well behaved, the alternative is to look for **algorithms that can do better than Hill Climbing**. The course looks at deterministic methods next (Part II: beam search, variable neighbourhood descent, tabu search) and at randomised methods later.

Module recap

- Hill Climbing: move to the best neighbour if better, else stop. Constant space, linear time, not complete; stops at local optima. It burns its bridges by not keeping OPEN.
- For maximisation it is steepest gradient ascent.
- 8-puzzle and Rubik's cube: along real solutions, heuristic values go up and down — local optima are unavoidable.
- Blocks world: h_1 (right block) leads HC to a local maximum Q; h_2 (right structure) is more discriminative and drives HC to the correct moves P then X. The slide's h_2 numbers mix two conventions; the ranking is the same either way.
- Heuristics define the terrain: global/local maxima, plateaus, ridges.

Part II — Escaping Local Optima: Deterministic Local Search

Module 4 Solution Space Search: SAT and TSP

4.1 Recap and plan

Hill Climbing looks only at local neighbours (constant space), moves only to a better node (linear time), and stops when no neighbour is better: it treats the problem as optimisation, is not complete, and may miss the global optimum. Heuristics that are monotonic and well behaved are hard to design, so we look for **algorithms that do better than Hill Climbing** — three deterministic ones in this part, randomised ones later. First, a change of viewpoint: searching in the **solution space**.

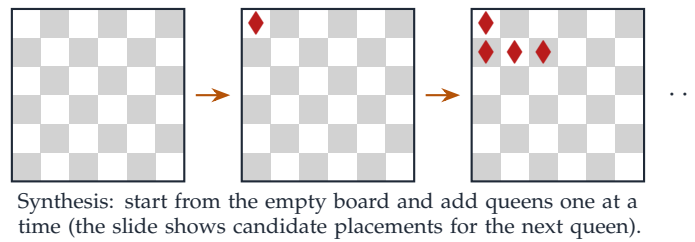
4.2 Solution space search

Definition: Solution space search

A formulation of the search problem such that **when we find the goal node we have the solution and we are done**: no path needs to be reconstructed. Every node in the search space is a **candidate solution**; a candidate is a solution if it satisfies the goal description.

- **Configuration problems** (N-queens, SAT, map colouring) fall easily into this category.
- **Planning problems** can be solved by solution space search too: each node is then a complete candidate *plan*, and we say we are searching in the **plan space**.

4.3 Synthesis vs perturbation



Definition: Synthesis vs perturbation

- **Synthesis** (constructive) methods start from the initial state and build the solution *piece by piece*. State space search of Week 2 is synthesis.
- **Perturbation** methods start from a *complete* candidate and produce new candidates by changing (perturbing) it. For N-queens, the representation $[a, c, e, b, f, d]$ gives, for each row i (the index), the column of its queen; **permuting the list** produces new candidates.

Formula / Derivation: Why the permutation representation is clever

In $[a, c, e, b, f, d]$ every row has exactly one queen (one list entry per row) and every column exactly one queen (it is a permutation). So row and column conflicts are *impossible by construction*; only diagonal conflicts remain. Queens in rows i and j attack diagonally iff

$$|\text{col}_i - \text{col}_j| = |i - j|.$$

The candidate space shrinks from $\binom{36}{6} = 1,947,792$ placements to $6! = 720$ permutations.

Check the slide's candidate $[a, c, e, b, f, d] = (1, 3, 5, 2, 6, 4)$: of the 15 pairs of rows, exactly two attack diagonally — rows 2 and 5 (columns 3 and 6: $|3 - 6| = 3 = |2 - 5|$) and rows 4 and 6 (columns 2 and 4: $|2 - 4| = 2 = |4 - 6|$). So it is a candidate, not a solution; a perturbative search would try to reduce this count of attacking pairs to 0. (No single swap of two entries reaches a solution from here — the best swaps leave one attacking pair.)

4.4 The SAT problem

Definition: SAT (Boolean satisfiability)

Given a Boolean formula over propositional variables $V = \{a, b, c, d, e, \dots\}$, each true or false (1 or 0), find an assignment that makes the formula evaluate to true (1).

Formula / Derivation: Example 1: an implication

$F = ((a \vee \neg e) \wedge (e \vee \neg c)) \supset (\neg c \vee \neg d)$ can be made true by $\{a = T, c = T, d = F, e = F\}$, among others:

$$(a \vee \neg e) = T, \quad (e \vee \neg c) = F \Rightarrow \text{antecedent} = F \Rightarrow F = T.$$

(Also the consequent $\neg c \vee \neg d = F \vee T = T$.) F is false only when the antecedent is true and $c = d = T$; the antecedent with $c = T$ forces $e = T$ and then $a = T$. So exactly one of the 16 assignments to a, c, d, e falsifies F : **15 of 16 satisfy it**.

Definition: Conjunctive Normal Form (CNF)

A conjunction ($\wedge$) of **clauses**, each a disjunction ($\vee$) of **literals** (a variable or its negation). SAT problems are very often studied in CNF. The slide's example has 5 variables and 6 clauses:

$$(b \vee \neg c) \wedge (c \vee \neg d) \wedge (\neg b) \wedge (\neg a \vee \neg e) \wedge (e \vee \neg c) \wedge (\neg c \vee \neg d).$$

Formula / Derivation: Solving the 5-variable CNF by reasoning

Unit clause $(\neg b)$ forces $b = 0$. Then $(b \vee \neg c)$ forces $c = 0$. Then $(c \vee \neg d)$ forces $d = 0$. Clauses $(e \vee \neg c)$ and $(\neg c \vee \neg d)$ are now satisfied. Only $(\neg a \vee \neg e)$ remains: a and e must not both be 1. Solutions (bits in order $a b c d e$):

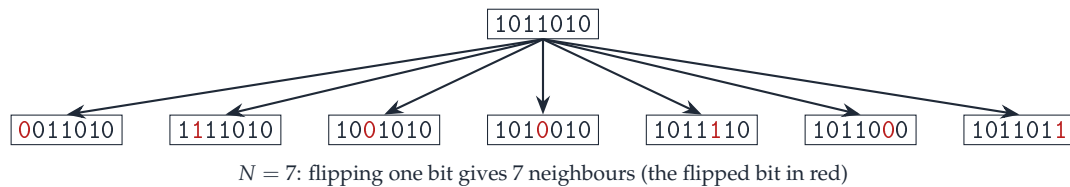
00000, 00001, 10000 (3 of the 32 candidates).

This chain of forced values is *unit propagation* — reasoning, not search. Local search does not use it; it relies on a heuristic instead (next section).

4.5 Solution space search and perturbative methods

Definition: Neighbourhood function

The **solution space** is the space of candidate solutions. A search method generates the neighbours of a candidate by applying some **perturbation** to it. The MoveGen function becomes a **neighbourhood function**. A SAT problem with N variables has 2^N candidates, each an N -bit string.



4.6 The Travelling Salesperson Problem

Definition: TSP

Given a set of cities and a distance between every pair, find a **Hamiltonian cycle** (visiting each city exactly once) of **least cost**. If the underlying graph is not completely connected, add the missing edges with very high cost — then **every permutation is a candidate**. The TSP is **NP-hard**. Benchmark instances with known optimal solutions are collected in **TSPLIB**.

4.6.1 Greedy constructive methods

Nearest Neighbour	Neigh-	Start at some city. Move to the nearest unvisited city, as long as it does not close the loop prematurely. <i>Variation</i> : extend the partial tour at either end.
Greedy (edge)		Sort all edges by length. Repeatedly add the shortest available edge, as long as it does not give a city degree 3 and does not close the loop prematurely.
Savings		Start with $n - 1$ tours of length 2 anchored on a base vertex and perform $n - 2$ merge operations. Each merge removes two edges to the base vertex (from two different loops) and adds one edge joining the two hanging vertices, choosing the pair with the maximal saving of total cost.

Formula / Derivation: The savings of a merge

With base vertex b , cities i and j are each on a “there-and-back” loop costing $2d(b, i)$ and $2d(b, j)$. Replacing the edges $b-i$ and $b-j$ by the single edge $i-j$ saves

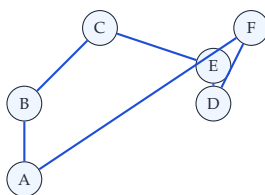
$$s(i, j) = d(b, i) + d(b, j) - d(i, j).$$

By the triangle inequality $s(i, j) \geq 0$. The Savings heuristic sorts pairs by $s(i, j)$ and merges greedily (Clarke and Wright, 1964).

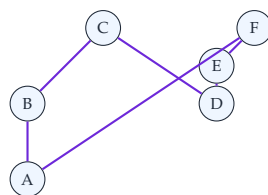
4.6.2 A worked example

Six cities (computed for these notes): A(0,1), B(0,3), C(2,5), D(5,3), E(5,4), F(6,5).

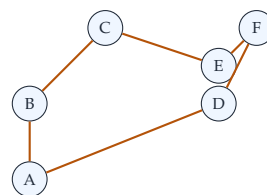
	A	B	C	D	E	F
A	–	2.00	4.47	5.39	5.83	7.21
B		–	2.83	5.00	5.10	6.32
C			–	3.61	3.16	4.00
D				–	1.00	2.24
E					–	1.41



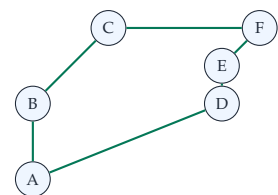
Nearest Neighbour from A
ABCEDF: 18.44



Greedy edge
ABCDEF: 18.06



Savings (base A)
ABCEFD: 17.03



Optimal
ABCFED: 16.63

Formula / Derivation: Tracing the three constructions

Nearest Neighbour from A: $A \rightarrow B$ (2.00) $\rightarrow C$ (2.83) $\rightarrow E$ (3.16) $\rightarrow D$ (1.00) $\rightarrow F$ (2.24), then forced back $F \rightarrow A$ (7.21). Total 18.44. The last edge is long: NN paints itself into a corner. (Starting from C or D, NN happens to find the optimum 16.63; from B it gives 19.20.)

Greedy: edges in order DE 1.00 ✓, EF 1.41 ✓, AB 2.00 ✓, DF 2.24 × (closes D–E–F), BC 2.83 ✓, CE 3.16 × (E already has degree 2), CD 3.61 ✓, CF, AC, BD, BE, AD, AE, BF × (degree), AF 7.21 ✓ closes the tour. Total 18.06.

Savings (base A): largest savings EF 11.63, DF 10.36, DE 10.22, CF 7.68, CE 7.14, CD 6.25, BC 3.64, Merge EF $\Rightarrow$ E–F; merge DF $\Rightarrow$ D–F–E; DE skipped (same route); CF skipped (F is now interior); merge CE $\Rightarrow$ C–E–F–D; CD skipped; merge BC $\Rightarrow$ B–C–E–F–D. Close through A: A–B–C–E–F–D–A = 17.03.

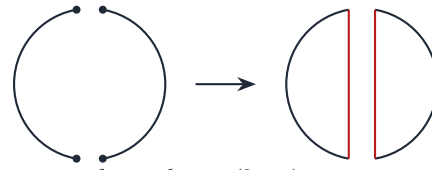
Note: What the slides show on 1000-city instances

The slides reproduce tours from D. S. Johnson's TSP challenge pages: the Greedy and Nearest Neighbour tours contain long "return" edges criss-crossing the map; the Savings tour is visibly closer to the Optimal tour. (The slide notes the source link is dead.) In the literature, typical excess over optimal on random instances is roughly 25% for NN, 15–20% for Greedy and about 10% for Savings. The worked example above shows the same ordering: NN > Greedy > Savings > optimal.

4.6.3 Perturbation operators for TSP



2-city exchange: two cities swap positions in the order. The four edges around them are replaced by the dashed ones. Choices: $\binom{n}{2}$.



2-edge exchange (2-opt): remove two edges, reconnect the other way. Choices: $\binom{n}{2}$ pairs of edges.

Definition: 3-edge exchange

Remove three edges and replace them by other edges. The three edges can be selected in $\binom{n}{3}$ ways, and the tour can be reconnected in **four** genuinely new ways (of the seven possible reconnections, three amount to single 2-edge exchanges). The **2-city exchange** changes four edges, so it is one case of the **4-edge exchange**.

Formula / Derivation: 2-opt on the worked example

A 2-edge exchange removes edges (a, b) and (c, e) and adds (a, c) and (b, e) , reversing the segment between them. It helps when

$$\Delta = d(a, c) + d(b, e) - d(a, b) - d(c, e) < 0.$$

Starting from the NN tour ABCEDF (18.44):

1. Remove ED, FA; add EF, DA: $\Delta = 1.41 + 5.39 - 1.00 - 7.21 = -1.41$. New tour ABCEFD, 17.03.
2. Remove CE, FD; add CF, ED: $\Delta = 4.00 + 1.00 - 3.16 - 2.24 = -0.40$. New tour ABCFED, **16.63 = optimal**.

No further 2-opt move improves it. Constructive heuristic + perturbative local search is the standard practical recipe.

Intuition: Why crossing edges are bad

Step 1 above removed the long edge FA that “crossed” the map. In the plane, if two tour edges cross, a 2-edge exchange that uncrosses them always shortens the tour (the triangle inequality on the two triangles formed at the crossing). So a 2-opt local optimum has no crossings — which is why good tours look like smooth loops.

4.7 Computational complexity and exploding search spaces

	SAT	TSP
Complexity class	NP-complete	NP-hard
Solving (brute force)	exponential time: 2^N candidates	factorial time: $N!$ candidates
Verifying a claimed answer	polynomial: evaluate the formula	not easy: “is this tour <i>optimal</i> ?” has no quick check

Both have huge search spaces — hence local search.

N	SAT: 2^N	TSP: $N!$	ratio $N!/2^N$
1	2	1	0.5
2	4	2	0.5
3	8	6	0.75
4	16	24	1.5
5	32	120	3.75
6	64	720	11.25
7	128	5,040	39.38
8	256	40,320	157.5
50	$1,125,899,906,842,624 \approx 1.13 \times 10^{15}$	3.04×10^{64}	2.7×10^{49}
100	1.27×10^{30}	9.33×10^{157}	7.4×10^{127}

Formula / Derivation: Worst-case time for SAT, real units

Inspecting a million candidates per second:

$$N = 50 : \frac{2^{50}}{10^6} = 1.126 \times 10^9 \text{ s} = \frac{1.126 \times 10^9}{3.156 \times 10^7} \text{ years} \approx \mathbf{35.7 \text{ years.}}$$

$$N = 100 : \frac{2^{100}}{10^6} = 1.27 \times 10^{24} \text{ s} \approx \mathbf{4.0 \times 10^{16} \text{ years}} \quad (\approx 3 \times 10^6 \text{ times the age of the universe}).$$

Exam trap: The slide's "decimal" time units

The slides convert the same numbers assuming 100 seconds in a minute, 100 minutes in an hour, 100 hours in a day and 1000 days in a year, giving "1.1 year" for $N = 50$ and " 1.27×10^{13} centuries" for $N = 100$. That is a deliberate simplification to make the powers of ten easy to follow; in real units $N = 50$ takes about 36 years. The point is unchanged: brute force is hopeless, and TSP ($N!$) is vastly worse than SAT (2^N).

Note: Distinct tours vs permutations

The slides count TSP candidates as $N!$ (all permutations). Distinct tours are $(N - 1)!/2$ (Week 2), because rotations and reversals give the same cycle. The factor $2N$ does not change the picture.

Module recap

- Solution space search: every node is a complete candidate; finding the goal ends the search (no path to reconstruct). Planning problems can be searched in plan space.
- Synthesis builds a solution piece by piece; perturbation modifies complete candidates via a neighbourhood function.
- SAT: 2^N bit strings; CNF; neighbours by flipping bits. TSP: Hamiltonian cycle of least cost; every permutation a candidate.
- Constructive TSP heuristics: Nearest Neighbour, Greedy edge, Savings $s(i, j) = d(b, i) + d(b, j) - d(i, j)$. Perturbations: 2-city exchange, 2-edge (2-opt), 3-edge exchange.
- SAT is NP-complete (2^N), TSP NP-hard ($N!$): hence local search.

Module 5 Deterministic Methods for Escaping Local Optima

5.1 Exploitation vs exploration

Algorithm: Hill Climbing (recap, general form)

```

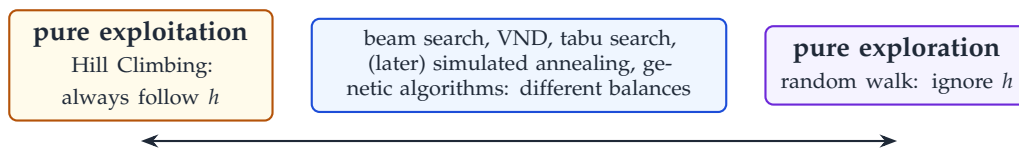
1:  $N \leftarrow \text{Start}$ 
2:  $\text{newNode} \leftarrow \text{best}(\text{MOVEGEN}(N))$ 
3: while  $\text{newNode}$  is better than  $N$  do
4:    $N \leftarrow \text{newNode}$ 
5:    $\text{newNode} \leftarrow \text{best}(\text{MOVEGEN}(N))$ 
6: end while
7: return  $N$ 

```

Definition: Exploitation and exploration

Hill Climbing **exploits** the gradient of $h(N)$: it always follows it. The gradient leads to the local optimum. Escaping local optima calls for *loosening the shackles of the heuristic function*: a tendency to **explore** as well.

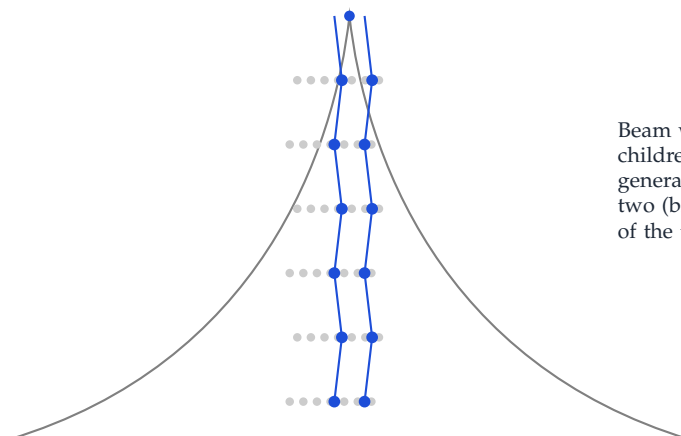
- **Exploitation** pulls the search along the gradient.
- **Exploration** leads it away from the trodden path.



5.2 Beam Search

Definition: Beam Search

Look at more than one option at each level: for a **beam width** w , keep the **best** w nodes at each level and expand only those. (After all, computer memory is becoming cheaper.) Width 1 is Hill Climbing; unbounded width is Breadth First Search.



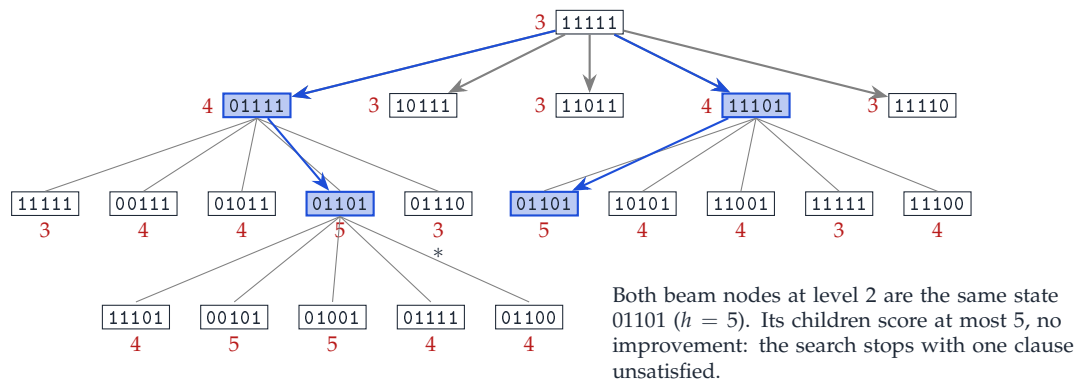
Beam width 2: at each level all children of the two kept nodes are generated (grey), and only the best two (blue) survive. A narrow slice of the tree is explored.

Formula / Derivation: Space and time of beam search

At each level at most w nodes are kept and $w \cdot b$ children evaluated. Over d levels: time $O(w b d)$, space $O(w b)$ — still **linear**, however deep the search goes. Beam search trades a constant factor w of extra work for a wider view than Hill Climbing.

5.2.1 Beam Search for SAT

Formula (5 variables $a b c d e$, 6 clauses): $(b \vee \neg c) \wedge (c \vee \neg d) \wedge (\neg b) \wedge (\neg a \vee \neg e) \wedge (e \vee \neg c) \wedge (\neg c \vee \neg d)$. Heuristic $h(n)$ = number of clauses satisfied — a **maximising** problem, goal $h = 6$. Neighbourhood: flip one bit. Start 11111.

**Note: Why beam search fails here, and the * node**

Level 1 keeps 01111 and 11101 ($h = 4$). Level 2: the best child of *both* is 01101 ($h = 5$); the slide keeps both copies, so the beam collapses to a single state. Every child of 01101 has $h \leq 5$, so the search terminates at a local maximum. Yet the node marked *, 01110 with $h = 3$, leads to a solution in three Hill Climbing steps (checked by computer): 01110 (3) $\rightarrow$ 01010 (4) $\rightarrow$ 00010 (5) $\rightarrow$ 00000 (6). “Starting with a value 3, the solution should be reached in 3 steps” — each step gains at least 1 and the maximum is 6. The heuristic simply rated the * node too low for the beam to keep it. **A wider beam is not the same as more exploration.**

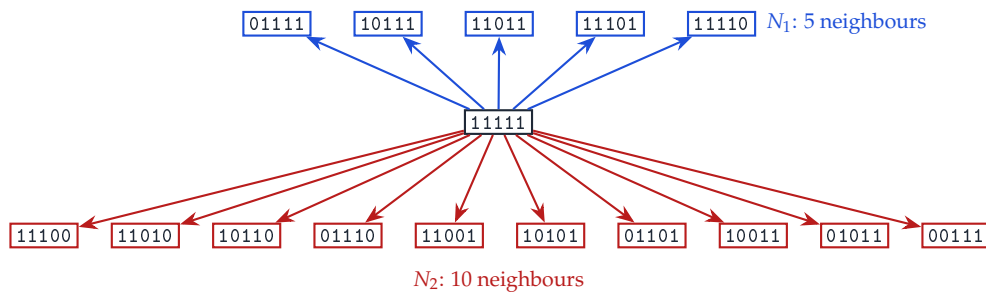
5.3 Different neighbourhood functions**Definition: Neighbourhood functions N_i**

For SAT with n variables, let N_i flip exactly i bits, N_{12} flip 1 or 2 bits, and N_{1-n} flip any of 1 to n bits. The denser the function, the more neighbours each candidate has.

Formula / Derivation: Neighbourhood sizes

$$|N_i| = \binom{n}{i}, \quad |N_{12}| = n + \binom{n}{2}, \quad |N_{1-n}| = \sum_{i=1}^n \binom{n}{i} = 2^n - 1.$$

For $n = 5$: $|N_1| = 5$, $|N_2| = 10$, $|N_{12}| = 15$, $|N_{1-5}| = 31$. The slide says N_{1-n} has 2^n neighbours; strictly it is $2^n - 1$ (every string except the current one). Either way, **inspecting them all amounts to brute force search.**



5.3.1 Effect of density on the performance of Hill Climbing

Hill Climbing inspects *all* neighbours and moves to the best one if it is better than the current node. It gets stuck on a local optimum when *none* of the neighbours is better.

- The **denser** the neighbourhood, the **less likely** this is (more neighbours, more chances that one is better). In the extreme, with N_{1-n} every candidate is a neighbour of every other, so the global optimum is always one step away and there are no local optima at all.
- But the denser the neighbourhood, the **more neighbours must be inspected at each step**. With N_{1-n} one step costs $2^n - 1$ evaluations: brute force.

Intuition: Local optimum is relative to the neighbourhood

“Local optimum” is not a property of a state alone; it is a property of a state *and* a neighbourhood function. A state can be a dead end under N_1 and have an improving move under N_2 . This is the idea behind the next algorithm.

5.4 Variable Neighbourhood Descent (VND)

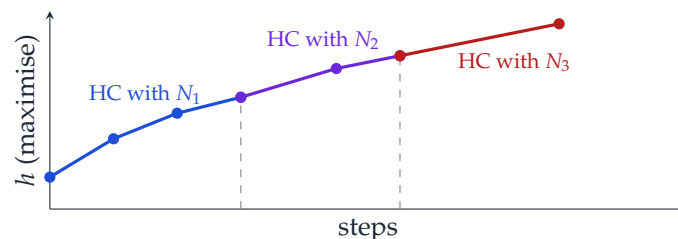
Algorithm: VariableNeighbourhoodDescent() — from the slide

```

1: node ← start
2: for i ← 1 to n do
3:   moveGen ← MOVEGEN(i)
4:   node ← HILLCLIMBING(node, moveGen)
5: end for
6: return node

```

The algorithm assumes that the function *moveGen* can be passed as a parameter, and that there are n moveGen functions sorted by the density of the neighbourhoods they produce. **The hope is that most of the “climbing” is done with the sparser (cheaper) neighbourhood functions**; the denser ones are called only to break out of the local optima of the sparse ones.



5.5 Best Neighbour: move anyway

Algorithm: Best Neighbour — a variation of Hill Climbing

```

1:  $N \leftarrow \text{Start}$ 
2:  $\text{bestSeen} \leftarrow N$ 
3: repeat
4:    $N \leftarrow \text{best}(\text{MOVEGEN}(N))$  ▷ move to the best neighbour anyway
5:   if  $N$  is better than  $\text{bestSeen}$  then
6:      $\text{bestSeen} \leftarrow N$ 
7:   end if
8: until some termination criterion
9: return  $\text{bestSeen}$ 

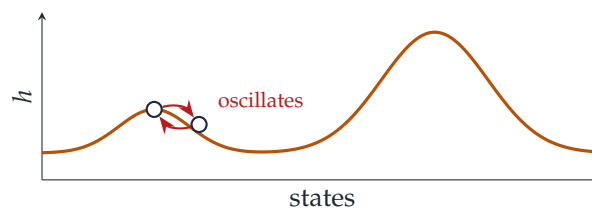
```

Note: Two changes from Hill Climbing

It moves even when the best neighbour is *worse*, so it does not stop at a local optimum; and because it no longer stops by itself, it needs an **external termination criterion** (a step budget, a time limit, or reaching $h = \text{goal value}$). It remembers the best node seen so far and returns that.

5.5.1 Getting off a local optimum — and straight back

Can Best Neighbour find a path to a better optimum? It *does* get off the optimum: from the peak it steps to the best (least bad) neighbour. **But it moves right back in the next cycle:** from that neighbour, the best neighbour is the peak it just left. It oscillates between two nodes forever.



5.6 Tabu Search: not allowed back immediately

Algorithm: Tabu Search — Best Neighbour restricted to allowed moves

```

1:  $N \leftarrow \text{Start}$ 
2:  $\text{bestSeen} \leftarrow N$ 
3: repeat
4:    $N \leftarrow \text{best}(\text{allowed}(\text{MOVEGEN}(N)))$  ▷ best neighbour anyway, but only if not tabu
5:   if  $N$  is better than  $\text{bestSeen}$  then
6:      $\text{bestSeen} \leftarrow N$ 
7:   end if
8: until some termination criterion
9: return  $\text{bestSeen}$ 

```

Note: The word

“Tabu” comes from the Tongan word for things that cannot be touched because they are sacred (the slide cites Wikipedia). A tabu move is one the search is forbidden to make for a while.

5.6.1 Tabu Search for SAT: the memory array

With neighbourhood N_1 (flip one bit), a move is identified by *which bit* it flips. Keep a memory array M recording **how many moves ago** each bit was changed (as a countdown). A **tabu tenure** tt defines the window within which a bit may not be changed again.

Formula / Derivation: Updating M

When bit k is flipped: $M_k \leftarrow tt$, and every other nonzero entry decreases by 1. A move that flips bit i is **allowed** iff $M_i = 0$.

The slide's illustration, $tt = 2, n = 7$:

Step	Candidate	Move	M after the move
0	1011010	—	0 0 0 0 0 0 0
1	1001010	flip bit 3	0 0 2 0 0 0 0 (bit 3 tabu for 2 cycles)
2	1001000	flip bit 6	0 0 1 0 0 2 0
3	0001000	flip bit 1	2 0 0 0 0 1 0 (bit 3 free again)

Intuition: Why a tenure breaks the oscillation

Best Neighbour bounced between two nodes because the move back simply undid the last flip. With $tt \geq 1$ that undo is forbidden, so the search is *forced* to try something else, and can walk across a plateau or down the side of a local peak towards a new region. Too small a tenure and it can still cycle (over longer loops); too large and it forbids so many moves that good ones are blocked. The tenure is a parameter tuned by experiment.

5.6.2 Aspiration criterion

Definition: Aspiration

If a *tabu* move would result in a node **better than bestSeen**, make an exception for it: add it to the allowed set. The tabu list exists to prevent cycling, and a move to a never-seen-before best value cannot be part of a cycle.

5.6.3 Driving the search into newer areas: frequency memory

Formula / Derivation: Frequency penalty (for a maximisation problem)

Maintain a frequency array $F = (F_1, \dots, F_n)$, initially all 0, counting **how many times each bit has been changed**. Let $node_i$ be the neighbour generated by changing bit i . Penalise each move in proportion to its frequency:

$$eval(node_i) \leftarrow eval(node_i) - F_i.$$

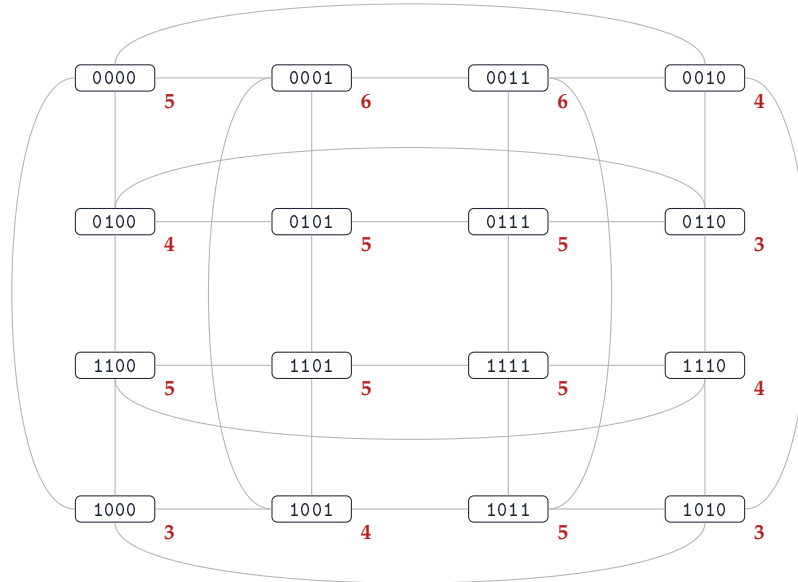
Bits that have been flipped often become less attractive, so the search is pushed to change bits it has rarely touched — **diversification** into unexplored regions. (M is short-term memory; F is long-term memory.)

5.7 A 4-variable SAT landscape

The formula

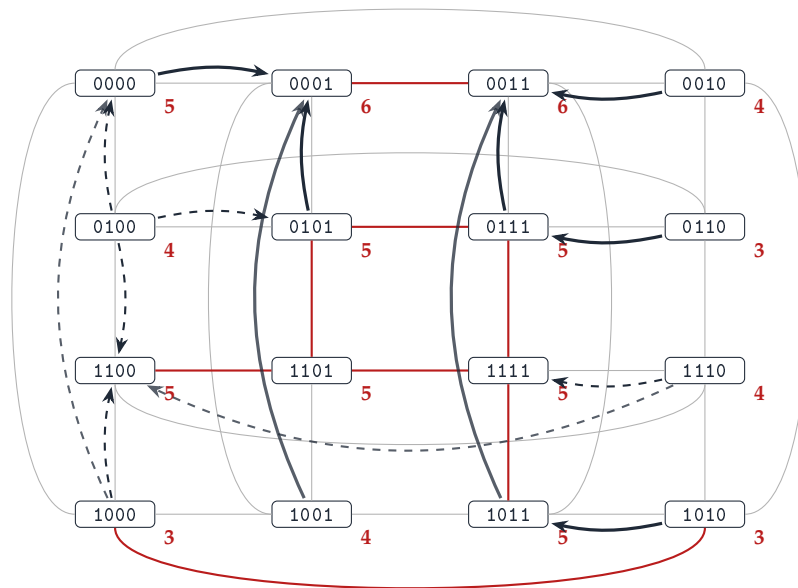
$$F = (a \vee \neg b) \wedge (\neg a \vee b \vee c) \wedge (\neg c \vee d) \wedge (d) \wedge (\neg a \vee b \vee d) \wedge (\neg a \vee \neg d),$$

with h = number of satisfied clauses and $\text{MoveGen} = N_1$. The 16 candidates form a 4-dimensional hypercube. The slides draw it as a tangle of lines; here it is laid out as a **Karnaugh map**: rows are $ab \in \{00, 01, 11, 10\}$, columns $cd \in \{00, 01, 11, 10\}$ (Gray-code order), so horizontally or vertically adjacent cells differ in exactly one bit. Curved edges on the border are the wrap-around neighbours (first and last row/column). Red numbers are h (all 16 match the slide; checked by computer).

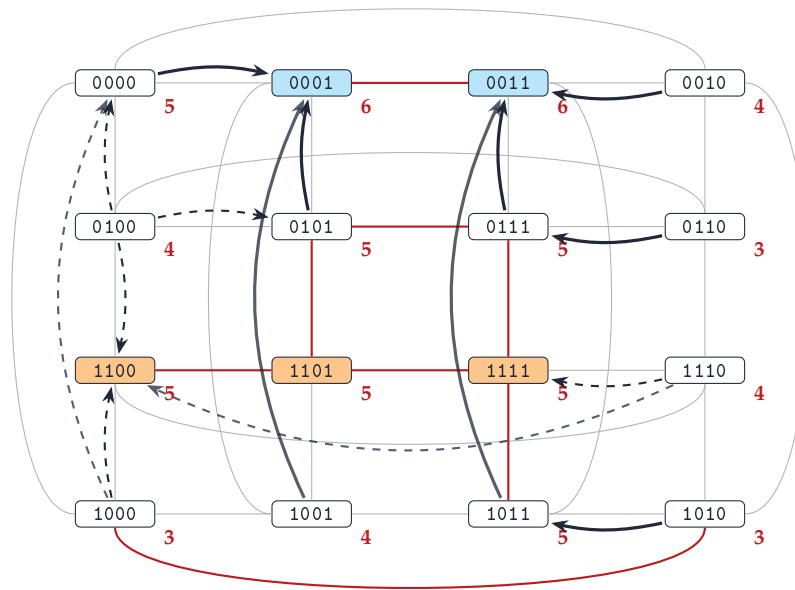


5.7.1 Gradients and ridges

Thick arrows show where Hill Climbing moves from each node: solid for a **unique best neighbour**, dashed for **one of multiple best neighbours**. Red edges join neighbours with *equal* h : these form the **ridges** and plateaus.



5.7.2 Maxima: global and local



Cyan: global maxima 0001 and 0011 ($h = 6$: all clauses satisfied). Orange: local maxima 1100, 1101, 1111 ($h = 5$, no neighbour better).

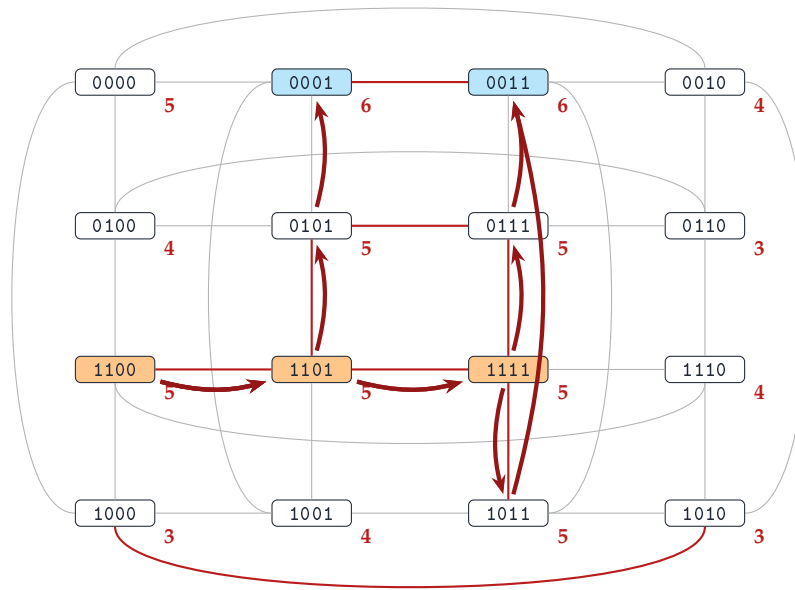
Formula / Derivation: Checking the local maxima

Node	h	neighbours (flip a, b, c, d) with h
1100	5	0100 (4), 1000 (3), 1110 (4), 1101 (5)
1101	5	0101 (5), 1001 (4), 1111 (5), 1100 (5)
1111	5	0111 (5), 1011 (5), 1101 (5), 1110 (4)

None has a neighbour with $h > 5$, so strict Hill Climbing stops at all three. They are connected by equal-valued edges (1100–1101–1111), and from 1101 and 1111 ridges lead to 0101, 0111, 1011, each of which has a better neighbour ($h = 6$). The global maxima are only two steps away — but the first step is sideways.

5.7.3 Tabu Search escapes

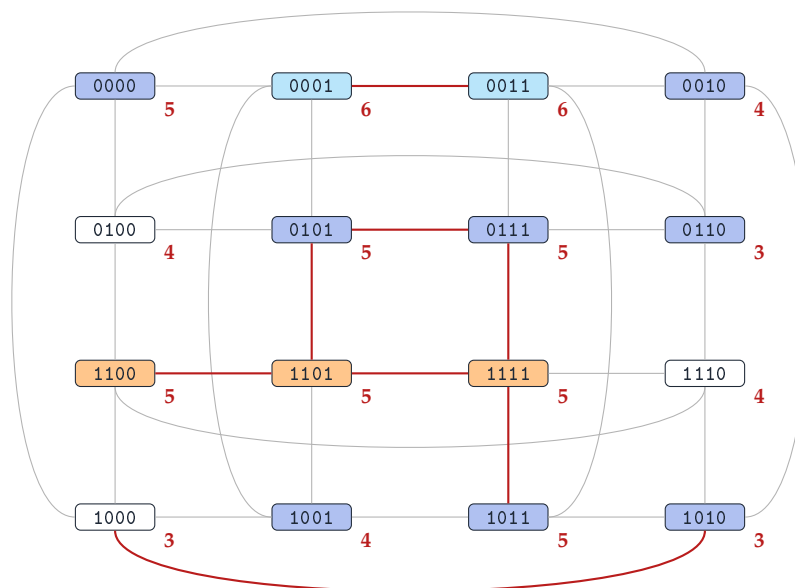
Starting from the local maximum 1100, Tabu Search (with $tt = 2$, choosing the best *allowed* neighbour even if it is not better) traverses one of three routes along the ridges to a solution (all three enumerated by computer):



Route	h along the way
1100 → 1101 → 0101 → 0001	5, 5, 5, 6
1100 → 1101 → 1111 → 0111 → 0011	5, 5, 5, 5, 6
1100 → 1101 → 1111 → 1011 → 0011	5, 5, 5, 5, 6

From 1100 the only neighbour with $h = 5$ is 1101 (flip d). At 1101, flipping d back to 1100 is *tabu*, so the search must choose between 0101 (flip a) and 1111 (flip c), both $h = 5$ — this tie is what creates the different routes. Plain Best Neighbour could have bounced back to 1100; the tabu memory prevents it. **Tabu Search can escape from local optima.**

5.8 Iterated Hill Climbing



Blue: starting points from which Hill Climbing is *guaranteed* to reach a solution (whatever the tie-breaking). They match the slide exactly.

The slide asks: **what is the sanctity of the start node?** Why not choose *different* starting points and run Hill Climbing from each?

Formula / Derivation: How often does Hill Climbing succeed from a random start here?

Of the 16 candidates: 2 are solutions; from 8 more (0000, 0010, 0101, 0110, 0111, 1001, 1010, 1011) Hill Climbing always reaches a solution; from 0100 and 1000 the outcome depends on tie-breaking (they can reach 0001 or stop at 1100); from 1110 it always stops at a local maximum (1100 or 1111); and 1100, 1101, 1111 are local maxima themselves. So a single random start succeeds with probability at least $p = 10/16 = 0.625$. With k independent random restarts,

$$P(\text{at least one success}) \geq 1 - (1 - p)^k = 1 - 0.375^k : \quad k = 3 : 0.947, \quad k = 5 : 0.993.$$

Iterated Hill Climbing — restart from random points and keep the best result — turns an unreliable algorithm into a reliable one cheaply. It introduces *randomness*, the subject of the next week (“watch this space...”).

Module recap

- Hill Climbing exploits the gradient; escaping local optima needs exploration.
- Beam search keeps the best w nodes per level; on the SAT example the beam collapses onto 01101 ($h = 5$) and fails, while a node rated only 3 leads to a solution.
- Neighbourhood density: $|N_i| = \binom{n}{i}$; denser means fewer local optima but costlier steps; N_{1-n} is brute force. VND climbs with sparse neighbourhoods first and denser ones only when stuck.
- Best Neighbour moves even to worse nodes (needs an external stopping rule) but oscillates. Tabu search forbids undoing recent moves (memory M , tenure tt), with aspiration (allow a tabu move that beats bestSeen) and frequency penalties $eval - F_i$ for diversification.
- On the 4-variable SAT landscape, 1100, 1101, 1111 are local maxima; tabu search escapes along ridges to 0001 or 0011; random restarts of Hill Climbing succeed with high probability.

Practice: Test Yourself

Work each problem before reading its solution. Tags give the module. All numerical answers were checked by running code.

Part A — Numerical problems and algorithm traces

Problem N1

[Module 2] 8-puzzle heuristics. Goal 1 2 3 / 8 _ 4 / 7 6 5. For the state 2 8 3 / 1 6 4 / 7 _ 5, compute h_1 (Hamming) and h_2 (Manhattan, tile by tile). Then compute both for each of its three successors. Which move would Best First choose?

Solution

Tiles out of place: 2 (at (1,1), goal (1,2)), 8 (at (1,2), goal (2,1)), 1 (at (2,1), goal (1,1)), 6 (at (2,2), goal (3,2)). $h_1 = 4$.

Manhattan: $d_2 = 1$, $d_8 = 1 + 1 = 2$, $d_3 = 0$, $d_1 = 1$, $d_6 = 1$, $d_4 = d_7 = d_5 = 0$. $h_2 = 5$.

Successors (blank at (3,2) can take a tile from above, left or right):

Move	h_1	h_2
6 moves down: 2 8 3 / 1 _ 4 / 7 6 5	3	4
7 moves right: 2 8 3 / 1 6 4 / _ 7 5	5	6
5 moves left: 2 8 3 / 1 6 4 / 7 5 _	5	6

Best First (either heuristic) chooses moving 6 down: it puts 6 in its goal cell.

Problem N2

[Module 2] Distance heuristics. The goal is at (1,2). For nodes $P = (4,6)$, $Q = (7,2)$ and $R = (5,5)$ compute the Euclidean and Manhattan heuristics. Do they rank the nodes the same way?

Solution

$P: \Delta = (3,4)$: Euclid 5, Manhattan 7. $Q: \Delta = (6,0)$: Euclid 6, Manhattan 6. $R: \Delta = (4,3)$: Euclid 5, Manhattan 7.

Euclid ranks $P = R (5) < Q (6)$. Manhattan ranks $Q (6) < P = R (7)$. **They disagree:** Q lies straight along an axis, where the two measures coincide, while P and R are diagonal, where Manhattan is larger by up to a factor $\sqrt{2}$. The choice of heuristic changes which node Best First picks.

Problem N3

[Modules 2–3] Best First vs Hill Climbing. Graph: S–A, S–B, A–C, A–D, B–E, C–F, E–G (goal), with h : S 9, A 5, B 7, C 3, D 4, E 2, F 6, G 0. (a) Trace Best First Search (OPEN sorted on h). (b) Trace Hill Climbing. (c) What does this show?

Solution

(a) OPEN: [S9] → pick S: [A5, B7] → pick A: [C3, D4, B7] → pick C: [D4, F6, B7] → pick D: [F6, B7] (D has no new children) → pick F: [B7] → pick B: [E2] → pick E: [G0] → pick G. Path

S-B-E-G. It explored the whole A-branch first because it looked closer, then recovered via B, still on OPEN.

(b) $S(9) \rightarrow A(5) \rightarrow C(3)$; C 's neighbours are $A(5)$ and $F(6)$, both worse: **stops at C**, a local minimum. Fails.

(c) The heuristic is misleading near A. Best First recovers because it keeps OPEN; Hill Climbing burnt its bridges.

Problem N4

[Module 3] Blocks world. Same goal as the slides (D, C, B, E, A bottom to top; F on the table). Compute h_1 and h_2 (blocks-below convention) for (a) every block on the table, (b) towers D-C-B-E and F-A.

Solution

(a) On the table: D and F are correct (+1 each), A, B, C, E are wrong (−1 each): $h_1 = 2 - 4 = -2$. Every block has 0 blocks below it: $h_2 = 0$.

(b) h_1 : A on F (wrong, −1), B on C +1, C on D +1, D +1, E on B +1, F +1: $h_1 = 4$.

h_2 : E has 3 correct blocks below (+3), B +2, C +1, D 0, A has 1 block below (F), wrong structure (−1), F 0: $h_2 = 5$. Moving A onto E then gives the goal ($h_1 = 6, h_2 = 10$).

Problem N5

[Module 4] N-queens representation. Check the candidate $[a, c, e, b, f, d] = (1, 3, 5, 2, 6, 4)$ for diagonal attacks. Is $(2, 4, 6, 1, 3, 5)$ a solution?

Solution

Queens in rows $i < j$ attack diagonally iff $|c_i - c_j| = j - i$. For $(1, 3, 5, 2, 6, 4)$: rows (2,5): $|3 - 6| = 3 = 3$ **attack**; rows (4,6): $|2 - 4| = 2 = 2$ **attack**; the other 13 pairs are safe. Two attacking pairs: not a solution.

For $(2, 4, 6, 1, 3, 5)$: check all 15 pairs, e.g. (1,2): $|2 - 4| = 2 \neq 1$; (1,4): $|2 - 1| = 1 \neq 3$; (3,4): $|6 - 1| = 5 \neq 1$; (4,5): $|1 - 3| = 2 \neq 1$; (5,6): $2 \neq 1$; (1,5): $1 \neq 4$; (2,6): $1 \neq 4$; ... none attack. **Yes**, it is one of the four 6-queens solutions.

Problem N6

[Modules 4–5] SAT heuristic. For the 5-variable CNF $(b \vee \neg c) \wedge (c \vee \neg d) \wedge (\neg b) \wedge (\neg a \vee \neg e) \wedge (e \vee \neg c) \wedge (\neg c \vee \neg d)$ (bits $abcde$), compute h (clauses satisfied) for 10101 and all its N_1 neighbours, then run Hill Climbing from it.

Solution

10101 ($a=1, b=0, c=1, d=0, e=1$): $(b \vee \neg c) = 0$, $(c \vee \neg d) = 1$, $(\neg b) = 1$, $(\neg a \vee \neg e) = 0$, $(e \vee \neg c) = 1$, $(\neg c \vee \neg d) = 1$: $h = 4$.

Neighbours: 00101 (5), 11101 (4), 10001 (5), 10111 (3), 10100 (4).

Hill Climbing (first best in bit order): 10101 (4) $\rightarrow$ 00101 (5) $\rightarrow$ 00001 (6). **Solved in 2 steps.** (00001 is one of the three solutions 00000, 00001, 10000.)

Problem N7

[Module 5] Neighbourhood sizes. For $n = 7$ and $n = 10$ variables compute $|N_1|$, $|N_2|$, $|N_{12}|$ and $|N_{1-n}|$.

Solution

$n = 7$: $|N_1| = 7$, $|N_2| = \binom{7}{2} = 21$, $|N_{12}| = 28$, $|N_{1-7}| = 2^7 - 1 = 127$.

$n = 10$: $|N_1| = 10$, $|N_2| = 45$, $|N_{12}| = 55$, $|N_{1-10}| = 1023$. (Also $|N_3| = \binom{10}{3} = 120$.) One step with N_{1-n} inspects every other candidate: brute force.

Problem N8

[Module 5] Tabu memory. $n = 7$, tabu tenure $tt = 3$, M initially all 0. The search flips bits 2, 5, 1, 7 in that order. Give M after each move and list which bits are tabu before the fifth move.

Solution

Rule: decrement every nonzero entry, then set the flipped bit to tt .

Flip	M after
2	0 3 0 0 0 0
5	0 2 0 0 3 0
1	3 1 0 0 2 0
7	2 0 0 0 1 0 3

Before the fifth move, bits 1, 5 and 7 are tabu ($M_i > 0$); bit 2 has just become free again after 3 moves.

Problem N9

[Module 5] Frequency penalty. In a maximisation problem the neighbours obtained by flipping bits 1–4 have $eval = 7, 8, 8, 6$, and the frequency array is $F = (0, 3, 1, 0)$. Which move is chosen with and without the penalty?

Solution

Without penalty: bit 2 or bit 3 (tie at 8). With $eval - F_i$: $7 - 0 = 7$, $8 - 3 = 5$, $8 - 1 = 7$, $6 - 0 = 6$. Now bits 1 and 3 tie at 7; the frequently flipped bit 2 drops to 5. The penalty steers the search away from bits it keeps changing.

Problem N10

[Module 4] TSP heuristics on the worked example. Cities A(0,1), B(0,3), C(2,5), D(5,3), E(5,4), F(6,5). (a) Run Nearest Neighbour from B. (b) Compute the savings $s(C, F)$ and $s(B, C)$ with base A. (c) For the tour ABCEFD, evaluate the 2-edge exchange that removes CE and FD.

Solution

(a) B→A (2.00) →C (4.47) →E (3.16) →D (1.00) →F (2.24) →B (6.32). Tour BACEDF, length **19.20** (worse than NN from A, 18.44; NN from C gives the optimum 16.63). The start city matters.

(b) $s(C, F) = d(A, C) + d(A, F) - d(C, F) = 4.47 + 7.21 - 4.00 = \mathbf{7.68}$; $s(B, C) = 2.00 + 4.47 - 2.83 = \mathbf{3.64}$.

(c) Add CF and ED: $\Delta = d(C, F) + d(E, D) - d(C, E) - d(F, D) = 4.00 + 1.00 - 3.16 - 2.24 = \mathbf{-0.40}$. Improving: new tour ABCFED of length $17.03 - 0.40 = 16.63$, the optimum.

Problem N11

[Module 4] Brute force times. A machine checks 10^9 candidates per second. How long does exhaustive search take for SAT with 40 variables, and for TSP with 20 cities (counting $N!$ permutations as the slides do)?

Solution

SAT: $2^{40} = 1.0995 \times 10^{12}$ candidates $\Rightarrow 1100 \text{ s} \approx$ **18 minutes**.
 TSP: $20! = 2.43 \times 10^{18} \Rightarrow 2.43 \times 10^9 \text{ s} \approx$ **77 years**. Even with half as many “cities” as SAT has variables, TSP is five orders of magnitude worse.

Problem N12

[Module 5] Iterated Hill Climbing. A problem has 1000 candidates; Hill Climbing succeeds from 150 of them. How many random restarts are needed to succeed with probability at least 0.99?

Solution

$p = 0.15$. Need $1 - (0.85)^k \geq 0.99$, i.e. $0.85^k \leq 0.01$, $k \geq \frac{\ln 0.01}{\ln 0.85} = \frac{-4.605}{-0.1625} = 28.3$. So **29 restarts**. Each run of Hill Climbing is short (linear), so this is cheap compared with exhaustive search.

Problem N13

[Module 5] Local maxima depend on the neighbourhood. In the 4-variable landscape, 1100 ($h = 5$) is a local maximum under N_1 . Is it still a local maximum under N_2 (flip exactly two bits)? Under N_3 ?

Solution

$N_2(1100)$: flip $ab \rightarrow 0000$ (5), $ac \rightarrow 0110$ (3), $ad \rightarrow 0101$ (5), $bc \rightarrow 1010$ (3), $bd \rightarrow 1001$ (4), $cd \rightarrow 1111$ (5). None exceeds 5: **still a local maximum** under N_2 (and hence under N_{12}).

$N_3(1100)$: flip $abc \rightarrow 0010$ (4), $abd \rightarrow$ **0001 (6)**, $acd \rightarrow 0111$ (5), $bcd \rightarrow 1011$ (5). **Not** a local maximum: a solution is one N_3 move away.

Lesson: whether a node is a local optimum depends on the neighbourhood function. VND would escape 1100 only when it reaches N_3 ; Tabu Search with N_1 escapes it in three cheap moves along the ridge instead.

Part B — Subjective questions with model answers**Question S1**

What is a heuristic function? How does Best First Search use it, and how does its termination criterion differ from Hill Climbing's?

Solution

A heuristic function $h(N)$, supplied by the user, estimates the distance from state N to the goal ($h(\text{goal}) = 0$). Best First Search keeps OPEN sorted on h (a priority queue) and always expands the candidate with the lowest h ; it stops when GoalTest succeeds (or OPEN empties). Hill Climbing keeps only the current node, moves to its best neighbour if that is better, and stops when no neighbour is better — an optimisation criterion that does not consult GoalTest, so it may stop at a local optimum.

Question S2

Compare the Hamming and Manhattan heuristics for the 8-puzzle. Why can both lead to local minima?

Solution

Hamming counts tiles out of place; Manhattan sums, over tiles, the grid distance to each tile's home. Each misplaced tile contributes at least 1 to Manhattan, so $\text{Manhattan} \geq \text{Hamming}$: it is more informed (it distinguishes a tile one step away from one four steps away). Both are computed tile by tile and ignore interactions: to swap two tiles you must temporarily move others away from home, increasing both heuristics. So along real solutions the values rise and fall (the slide's plot: 4,3,4,4,3,2,1,0), and states exist where every move increases h — local minima.

Question S3

Explain the properties of Best First Search: sense of direction, space, completeness and solution quality.

Solution

Direction: it expands nodes that look closest to the goal, so it heads towards the goal rather than following a fixed trajectory. Space: its frontier depends on the heuristic — small with a good h , up to BFS-like with a poor one. Completeness: on a finite graph it is complete, because it never discards candidates; if h leads into a dead end it resumes from the best remaining node on OPEN. On infinite graphs a misleading h can trap it. Quality: it ignores the cost already incurred, so it may return a path longer than the shortest (the slide's five-move example).

Question S4

Why is Hill Climbing called a constant-space, linear-time algorithm? What does it sacrifice?

Solution

It stores only the current node and examines its neighbours: space independent of the search depth. Each move strictly improves h , so it never revisits a node and the number of moves is bounded by the length of an improving path (at most $h(s_0)$ steps if h is a non-negative integer). It sacrifices completeness and optimality: with no OPEN list it cannot return to alternatives ("burnt its bridges"), so at a local optimum, plateau or ridge it simply stops, possibly far from the solution.

Question S5

Using the blocks world example, explain why h_2 is "more discriminative" than h_1 .

Solution

h_1 scores each block ± 1 by what it rests on. From the start it rates moving A onto E highest (state Q, $h_1 = 4$), because A is then on the right block — but E is on the wrong block, and every move from Q lowers h_1 : a local maximum. h_2 scores each block $\pm n$ by whether the whole structure beneath it is correct, weighted by its size. It rates moving A to the table (P) highest, since that removes A from a large wrong structure, and from P it rates E onto B highest, building a correct tower. Hill Climbing with h_2 reaches the goal. h_2 distinguishes

states h_1 treats as equal (S, P, R, T all have $h_1 = 2$) and reflects the true difficulty better.

Question S6

What is solution space search? Contrast synthesis and perturbation.

Solution

Solution space search formulates the problem so that each node is a complete candidate solution; finding a node satisfying the goal description ends the search, with no path to reconstruct. Configuration problems fit naturally; planning problems can be searched in plan space. Synthesis (constructive) methods build a solution from the initial state piece by piece, as in state space search or greedy TSP construction. Perturbation methods start from a complete candidate and move to neighbours by small changes (flip a bit, swap two queens, exchange two edges), guided by a heuristic evaluation.

Question S7

Describe the Nearest Neighbour, Greedy and Savings heuristics for TSP and their typical weaknesses.

Solution

Nearest Neighbour starts at a city and repeatedly moves to the nearest unvisited city; it tends to leave a long final edge back to the start. Greedy sorts all edges and adds the shortest one that keeps degrees ≤ 2 and closes no premature cycle; it produces good local pieces joined by some long edges. Savings starts with $n - 1$ there-and-back tours from a base city and performs $n - 2$ merges, each replacing two base edges by one connecting edge with maximal saving $s(i, j) = d(b, i) + d(b, j) - d(i, j)$; it usually gives the best tours of the three. All are fast constructive methods whose output is typically improved by perturbation (2-opt, 3-opt).

Question S8

Explain the exploration–exploitation trade-off and how beam search, VND, Best Neighbour and Tabu Search each add exploration.

Solution

Exploitation follows the heuristic gradient; exploration moves away from it to discover other regions. Beam search keeps w good nodes per level rather than one, widening the view (but still ranked by h). VND changes the neighbourhood function, switching to denser neighbourhoods when stuck so that more distant candidates become reachable. Best Neighbour moves to the best neighbour even if worse, so it leaves local optima (but oscillates). Tabu Search adds memory forbidding recent moves, so it cannot immediately return, and can traverse plateaus and ridges; aspiration and frequency penalties refine this.

Question S9

Why does beam search with width 2 fail on the slide's SAT example?

Solution

From 11111 ($h = 3$) the beam keeps 01111 and 11101 ($h = 4$). The best child of both is the same node, 01101 ($h = 5$); both copies are kept, so the beam collapses onto one state. All

neighbours of 01101 have $h \leq 5$, so no improvement is possible and the search stops one clause short. Meanwhile 01110, rated only 3 at level 2, leads to a solution in three steps. Beam search still ranks by h , so a misleading heuristic can exclude the right nodes from the beam; duplicate nodes waste beam slots.

Question S10

Explain how the density of the neighbourhood function affects Hill Climbing, and the idea of VND.

Solution

A local optimum is a node with no better neighbour. Denser neighbourhoods (more bits flipped) give each node more neighbours, so it is less likely that none is better; at the extreme N_{1-n} every candidate neighbours every other and no local optima exist. But each step then inspects more neighbours: $\binom{n}{i}$ for N_i , $2^n - 1$ for N_{1-n} , which is brute force. VND runs Hill Climbing with N_1 , then from the result with N_2 , and so on, so that most climbing uses cheap sparse neighbourhoods and the expensive dense ones are used only to escape local optima.

Question S11

Describe Tabu Search: memory array, tabu tenure, aspiration and frequency penalties.

Solution

Tabu Search always moves to the best *allowed* neighbour, even if worse, and returns the best node seen (it needs an external stopping rule). For bit-flip neighbourhoods, a memory array M records for each bit how many moves remain before it may be flipped again; after flipping bit k , $M_k \leftarrow tt$ (the tabu tenure) and other entries count down. This prevents immediately undoing moves, so the search cannot oscillate and can cross plateaus and ridges. Aspiration: a tabu move is allowed if it would produce a node better than bestSeen. Frequency memory F counts how often each bit has been flipped; penalising $eval(node_i) - F_i$ drives the search into less explored regions.

Question S12

Using the 4-variable SAT landscape, explain ridges, local maxima and how Tabu Search and Iterated Hill Climbing succeed where Hill Climbing fails.

Solution

With h = satisfied clauses and N_1 , nodes 1100, 1101, 1111 have $h = 5$ and no better neighbour: local maxima. They are linked by equal-valued edges (a ridge) to 0101, 0111, 1011 ($h = 5$), each adjacent to a solution (0001 or 0011, $h = 6$). Hill Climbing started at or drawn into the ridge stops. Tabu Search from 1100 must move to 1101 and cannot flip back, so it proceeds along the ridge (via 0101, or via 1111 then 0111 or 1011) to a solution. Iterated Hill Climbing instead restarts from random nodes: from 10 of the 16 nodes Hill Climbing reaches a solution, so a few restarts almost certainly succeed.

Part C — Multiple-choice questions

Answers and explanations follow the questions.

- Q1.** DFS and BFS are called blind because they:
(a) cannot detect the goal (b) follow the same trajectory regardless of where the goal is (c) use no memory (d) are randomised
- Q2.** A heuristic function $h(N)$ estimates:
(a) the cost from start to N (b) the distance from N to the goal (c) the number of children of N (d) the depth of N
- Q3.** In Best First Search, OPEN is a:
(a) stack (b) queue (c) priority queue sorted on h (d) set
- Q4.** The one-line change that turns DFS into Best First Search is:
(a) $\text{OPEN} \leftarrow \text{append}(\text{tail OPEN}, \text{NEW})$ (b) $\text{OPEN} \leftarrow \text{sort}_h(\text{append}(\text{NEW}, \text{tail OPEN}))$ (c) remove CLOSED (d) add a depth bound
- Q5.** The Hamming heuristic for the 8-puzzle counts:
(a) moves made so far (b) tiles out of place (c) sum of tile distances (d) inversions
- Q6.** For any 8-puzzle state, $h_{\text{Manhattan}}$ and h_{Hamming} satisfy:
(a) $h_{\text{Man}} \leq h_{\text{Ham}}$ (b) $h_{\text{Man}} \geq h_{\text{Ham}}$ (c) they are equal (d) no relation
- Q7.** For goal $(0,0)$, the Manhattan heuristic of node $(6,4)$ is:
(a) 7.21 (b) 10 (c) 24 (d) 2
- Q8.** The Euclidean heuristic of node $(8,5)$ for goal $(0,0)$ is:
(a) 13 (b) 9.43 (c) 8.42 (d) 89
- Q9.** Best First Search can recover from a dead end because:
(a) it backtracks one step (b) it keeps the other candidates on OPEN (c) it restarts (d) it uses a tabu list
- Q10.** Best First Search is guaranteed to find:
(a) the shortest path (b) a path, on a finite graph (c) the optimal tour (d) nothing
- Q11.** Hill Climbing terminates when:
(a) the goal is found (b) no neighbour is better than the current node (c) OPEN is empty (d) a depth bound is reached
- Q12.** The space complexity of Hill Climbing is:
(a) exponential (b) linear in depth (c) constant (d) quadratic
- Q13.** “Hill Climbing has burnt its bridges” refers to:
(a) it deletes CLOSED (b) it does not store OPEN (c) it cannot move uphill (d) it uses random restarts
- Q14.** In the blocks world, $h_1(\text{start})$ equals:
(a) -1 (b) 2 (c) 4 (d) 6
- Q15.** According to h_1 , the best move from the start leads to state:
(a) P (b) Q (c) R (d) T
- Q16.** From h_1 's perspective, state Q is:
(a) the goal (b) a local maximum (c) a plateau (d) a ridge
- Q17.** According to h_2 , the best move from the start is:
(a) A onto E (b) A to the table (c) E onto A (d) E to the table

- Q18.** A plateau in a heuristic landscape is a region where:
 (a) h is maximal (b) neighbours have equal h (c) h decreases steadily (d) no neighbours exist
- Q19.** In solution space search:
 (a) the path must be reconstructed (b) every node is a candidate solution (c) only planning problems apply (d) MoveGen is not used
- Q20.** The N-queens representation $[a, c, e, b, f, d]$ makes impossible:
 (a) diagonal attacks (b) row and column attacks (c) all attacks (d) nothing
- Q21.** A SAT problem with N variables has how many candidates?
 (a) N (b) N^2 (c) 2^N (d) $N!$
- Q22.** The unit clause $(\neg b)$ in a CNF formula:
 (a) can be ignored (b) forces $b = 0$ (c) forces $b = 1$ (d) makes the formula unsatisfiable
- Q23.** With $N = 7$ and a one-bit-flip neighbourhood, the number of neighbours is:
 (a) 1 (b) 7 (c) 21 (d) 128
- Q24.** TSP is:
 (a) in P (b) NP-complete and easy to verify (c) NP-hard (d) undecidable
- Q25.** The Savings heuristic starts with:
 (a) the shortest edge (b) $n - 1$ tours of length 2 anchored on a base vertex (c) a random tour (d) a minimum spanning tree
- Q26.** The saving of merging cities i and j with base b is:
 (a) $d(i, j)$ (b) $d(b, i) + d(b, j) - d(i, j)$ (c) $d(b, i) - d(b, j)$ (d) $d(b, i)d(b, j)$
- Q27.** The number of ways to choose two edges for a 2-edge exchange is (per the slides):
 (a) n (b) $\binom{n}{2}$ (c) $\binom{n}{3}$ (d) $n!$
- Q28.** The 2-city exchange is a special case of:
 (a) the 2-edge exchange (b) the 3-edge exchange (c) the 4-edge exchange (d) the savings merge
- Q29.** With a million candidates per second, brute-force SAT with 50 variables takes, in real time, about:
 (a) 1.1 years (b) 36 years (c) 1 day (d) 10^{13} centuries
- Q30.** Beam search with beam width 1 is:
 (a) BFS (b) DFS (c) Hill Climbing (d) Tabu search
- Q31.** In the slide's beam search on SAT, the search fails because:
 (a) the beam is too wide (b) both beam slots end up holding the same node with no better neighbour (c) the formula is unsatisfiable (d) h is not defined
- Q32.** For $n = 5$, N_{12} has how many neighbours?
 (a) 5 (b) 10 (c) 15 (d) 31
- Q33.** Using N_{1-n} in Hill Climbing amounts to:
 (a) random walk (b) brute force search (c) tabu search (d) beam search
- Q34.** VND hopes that most of the climbing is done with:
 (a) the densest neighbourhood (b) sparser neighbourhood functions (c) random restarts (d) a tabu list

- Q35.** The Best Neighbour algorithm needs:
 (a) a priority queue (b) an external termination criterion (c) a heuristic of zero (d) CLOSED
- Q36.** The problem with Best Neighbour at a local optimum is that it:
 (a) cannot move (b) moves off and then straight back (c) always finds the goal (d) uses exponential space
- Q37.** With tabu tenure $tt = 2$, after flipping bit 3 then bit 6 (from $M = 0000000$), M is:
 (a) 0020020 (b) 0010020 (c) 0020010 (d) 0000020
- Q38.** The aspiration criterion allows a tabu move when:
 (a) it is the only move (b) it produces a node better than bestSeen (c) the tenure is 0 (d) always
- Q39.** In the frequency-based tabu variant (maximisation), the evaluation is modified as:
 (a) $eval + F_i$ (b) $eval - F_i$ (c) $eval \times F_i$ (d) $eval / F_i$
- Q40.** In the 4-variable SAT landscape, the global maxima are:
 (a) 1100 and 1111 (b) 0001 and 0011 (c) 0000 and 1000 (d) 1101 only

Answers and explanations

Q	Ans	Explanation
1	b	Oblivious of the goal: same predetermined trajectories.
2	b	An estimate of the distance to the goal; $h(\text{goal}) = 0$.
3	c	Candidates are kept in order of h .
4	b	Sort (or merge) on h after adding new nodes.
5	b	Tiles out of place (the blank is not counted).
6	b	Each misplaced tile adds at least 1 to Manhattan.
7	b	$6 + 4 = 10$ (7.21 is the Euclidean value).
8	b	$\sqrt{64 + 25} = \sqrt{89} = 9.43$.
9	b	OPEN still holds the next best candidates.
10	b	Complete on finite graphs; no shortest-path guarantee.
11	b	Optimisation-style termination.
12	c	Only the current node and its neighbours.
13	b	It keeps no OPEN list to fall back on.
14	b	$(-1) + 1 + 1 + 1 + (-1) + 1 = 2$.
15	b	In Q, A is on E, so $val_A = +1$ and $h_1 = 4$.
16	b	Every neighbour of Q has $h_1 = 2 < 4$.
17	b	P (A to the table) has the highest h_2 under either convention.
18	b	No gradient to follow.
19	b	Finding the goal node is the solution.
20	b	One queen per row (list entry) and per column (permutation).
21	c	Each variable 0 or 1.
22	b	A unit clause fixes its literal.
23	b	One neighbour per bit.
24	c	NP-hard; verifying optimality is not easy.
25	b	Then $n - 2$ merges.

Q	Ans	Explanation
26	b	The two base edges removed minus the new edge added.
27	b	As stated on the slide.
28	c	Swapping two cities changes four edges.
29	b	$1.126 \times 10^9 \text{ s} \approx 35.7 \text{ years}$; “1.1 year” uses the slide’s decimal units.
30	c	Keep only the best node: Hill Climbing (with its stopping rule).
31	b	Both keep 01101 ($h = 5$), a local maximum.
32	c	$5 + 10 = 15$.
33	b	$2^n - 1$ neighbours: every candidate.
34	b	Dense neighbourhoods only to escape.
35	b	It no longer stops by itself.
36	b	Oscillation between two nodes.
37	b	Bit 3 counts down to 1, bit 6 set to 2.
38	b	A new best cannot be part of a cycle.
39	b	Penalise frequently flipped bits.
40	b	All six clauses satisfied.

Next: randomised methods — stochastic local search, simulated annealing, and population-based methods such as genetic algorithms and ant colony optimisation.